

Dossier de certification — Bloc 2 : Concevoir et développer des applications logicielles

Projet : MyMemoMaster — plateforme web de révision et de suivi pédagogique

Candidat : Macabiau Frédéric — Expert en développement logiciel (RNCP niveau 7)

Plan du dossier

Section	Contenu	Compétence couverte
Introduction	Présentation du projet	—
1	Environnements de développement, de test et de déploiement	C2.1.1
2	Intégration continue	C2.1.2
3	Prototypage et conception de l'application	C2.2.1
4	Harnais de tests unitaires	C2.2.2
5	Développement : évolutivité, sécurisation, accessibilité	C2.2.3
6	Déploiement continu et progressif	C2.2.4
7	Cahier de recettes	C2.3.1
8	Plan de correction des bogues	C2.3.2
9	Documentation technique d'exploitation	C2.4.1
10	Glossaire	—
Annexes	A. Galerie du prototype · B. Captures de l'application · C. Index des documents du dépôt · D. Sources bibliographiques · E. Synthèse de couverture des compétences	—

Présentation du projet

La plupart des étudiants ont des méthodes de révision peu efficaces qui les mettent en difficulté : 83,6 % s'appuient sur des méthodes de révision passives, 27 % préparent leurs examens au dernier moment, seulement 34 % disposent d'un calendrier de révision utile et seul 1 étudiant sur 2 s'entraîne via des annales ou des exercices (études et enquêtes versionnées dans[docs/sources/] (docs/sources/) – voir Annexe D). Pour remédier à ces problèmes, MyMemoMaster propose une plateforme de révision et de suivi étudiant tout-en-un, à destination des étudiants principalement

et des enseignants. La plateforme centralise des fonctionnalités fondées sur des méthodes pédagogiques actives dont l'efficacité est documentée par ces mêmes travaux :

- **Systèmes de Leitner** : questions-réponses à répétition espacée — l'algorithme représente les cartes dans des « boîtes » et fait remonter plus fréquemment les questions échouées ;
- **Cartes mentales** : éditeur graphique de schémas de notions et de leurs liens ;
- **Séries d'exercices** : entraînement avec correction automatique, y compris une correction par **similarité sémantique** pour les réponses ouvertes, exécutée par un modèle d'IA embarqué dans l'API (bibliothèque `@xenova/transformers`, modèle `all-mpnet-base-v2`);
- **Calendrier, échéances et rappels** : planification des sessions de révision, todo-list, notifications par email (file de traitement BullMQ/Redis) ;
- **KPI personnels et pédagogiques** : indicateurs de progression pour l'étudiant, tableaux de bord pour l'enseignant, avec gestion du consentement de partage ;
- **Groupe classes et établissements** : partage de ressources pédagogiques, invitations, périmètre d'administration pour les gérants d'établissement.

Cette liste correspond aux modules effectivement présents dans le code : chaque fonctionnalité ci-dessus a son module de routes monté dans `my_memo_master_api/app.js` (lignes 111–146 : `leitnerSystemRoutes`, `diagrammeRoutes`, `questionRoutes`, `semanticRoutes`, `calendarEventRoutes`, `kpiRoutes`, `classGroupRoutes`, `etablissementRoutes`...).

Stack technique

La stack est documentée dans `docs/CONVENTIONS.md` et vérifiable dans les manifestes npm de l'api (`my_memo_master_api/package.json`) et celui du front (`my_memo_master_front/package.json`) :

Couche	Technologie
Runtime API	Node.js 22
Framework API	Express.js v4
ORM / Base de données	Sequelize v6 — PostgreSQL 15 (test/preprod/prod), SQLite (développement local)
Authentification	JWT (<code>jsonwebtoken</code>) + <code>bcryptjs</code> , refresh token avec rotation
File de tâches asynchrones	BullMQ + Redis (rappels, notifications email)
IA embarquée	<code>@xenova/transformers</code> (correction par similarité sémantique)
Front	Vue.js 3 + Vite v6 + Pinia (state) + Tailwind CSS v3
Tests	Jest + Supertest (API), Vitest + <code>@vue/test-utils</code> (front)
Qualité	ESLint (API v9, front v8) + Prettier
Documentation API	swagger-jsdoc + swagger-ui-express

Couche	Technologie
Conteneurisation	Docker, Docker Compose, Helm/Kubernetes
CI/CD	GitHub Actions

Organisation du depot

```

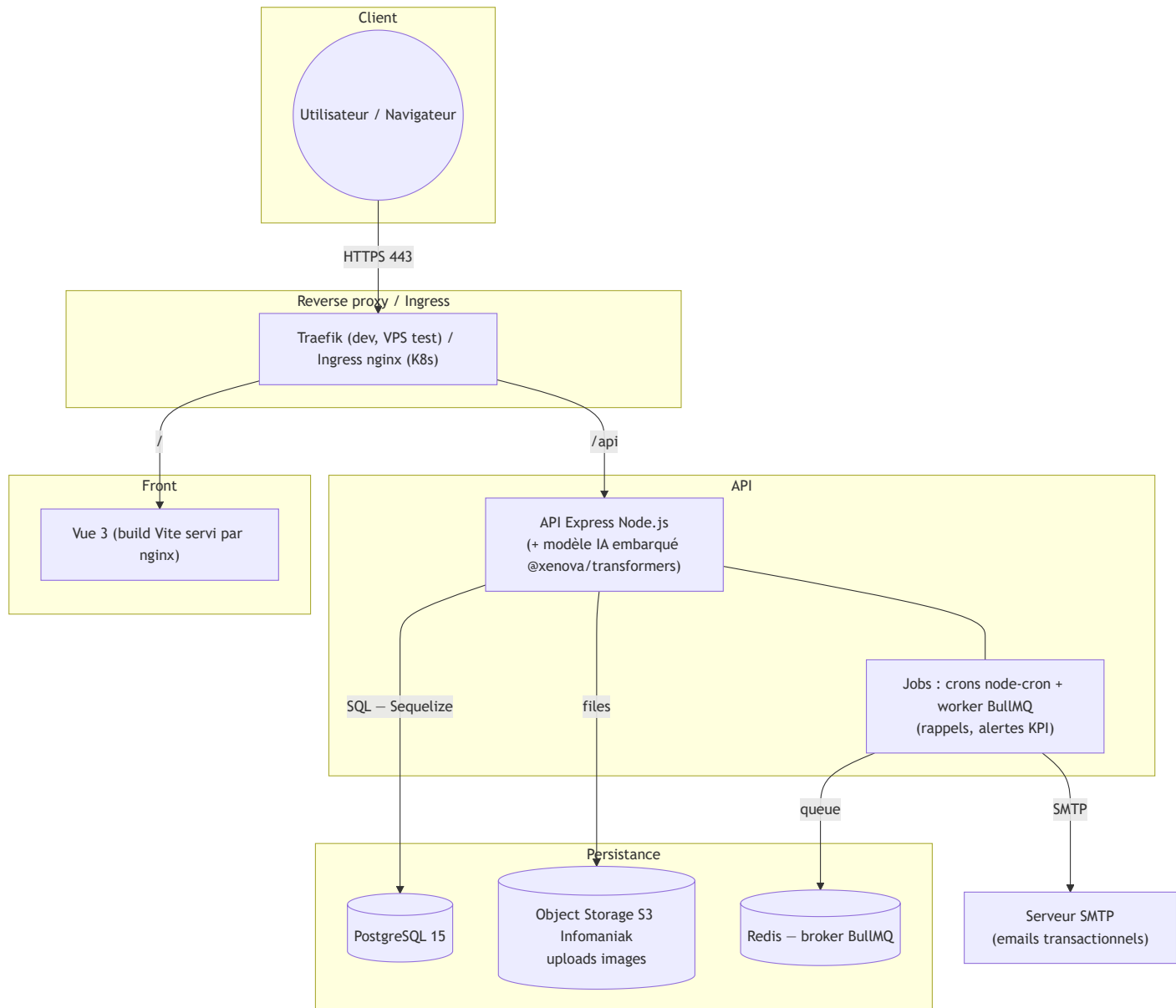
MyMemoMaster/
├── my_memo_master_api/      # API Express (controllers, services, models, tests...)
├── my_memo_master_front/   # Front Vue 3 (pages, components, stores, tests...)
├── .github/workflows/      # Pipelines CI/CD (ci.yml, cd.yml, notify_ci.yml)
├── helm/                   # Chart Helm (déploiements Kubernetes preprod/prod)
├── k8s/                    # Manifests Kubernetes + scripts de migration Helm
├── server_docker_compose/  # docker-compose déployé sur le VPS de test
├── docker-compose.yml      # Environnement de développement local complet
├── docs/                   # Documentation : manuels (déploiement, utilisation, RUNBOOK),
audits (OWASP, RGAA), prototype + captures, sources bibliographiques, synthèse mémoire du projet
└── diagrams/              # Spécifications : règles métier, schéma BDD, UI

```

La mémoire du projet est synthétisée dans [docs/MEMOIRE_PROJET.md](#), qui présente le dispositif et renvoie vers les fichiers sources de [.agents/](#). Le choix du monorepo est une décision de conception, argumentée en section 1.4.

Architecture de l'application

Le schéma ci-dessous représente l'architecture de l'application, telle qu'elle résulte des fichiers d'infrastructure du dépôt ([docker-compose.yml](#), [helm/templates/](#)). Légende : les rectangles sont des conteneurs applicatifs, les cylindres des services de persistance, les flèches indiquent le sens des requêtes, et chaque sous-graphe correspond à une couche d'infrastructure.



Deux précisions d'honnêteté documentaire : contrairement à l'architecture **cible** décrite dans [archi.md](#), l'API IA séparée (Python/FastAPI) n'existe pas à ce jour — la correction sémantique s'exécute dans le processus Node de l'API ([my_memo_master_api/services/Semantic.service.js](#)) ; et Redis sert **exclusivement** de broker BullMQ, pas de cache applicatif (règle de [.agents/CONVENTIONS.md](#)).

Section 1 — Environnements de développement, de test et de déploiement

Compétence couverte : C2.1.1 — Mettre en œuvre des environnements de déploiement et de test en y intégrant les outils de suivi de performance et de qualité afin de permettre le bon déroulement de la phase de développement du logiciel.

1.1 Environnement de développement local

J'ai conçu l'environnement de développement pour qu'un contributeur soit opérationnel rapidement : copier `.env.example` en `.env`, Ajouter les URLS et clefs d'accès aux services externes utilisés par l'app (S3, serveur SMTP, token cloudflare) puis `docker compose up --build`. La procédure complète est documentée dans le [README.md](#) (il y a également une partie 2, « Bien commencer »), qui liste aussi l'outillage attendu (VS Code, Postman, Git, Docker).

Un docker-compose unifié piloté par profils

Plutôt que d'avoir plusieurs docker compose divergents, j'ai unifié les environnements Docker dans un seul [docker-compose.yml](#) piloté par les *profiles*. Les profils sont documentés dans L'en-tête du fichier :

```
# Usage:
#   Dev local : docker compose --env-file .env.dev up --build
#   Test VPS  : docker compose --env-file .env.test up
#   Prod     : docker compose --env-file .env.prod up
#
# Profiles (défini via COMPOSE_PROFILES dans le .env.*) :
#   dev  - Traefik local HTTP, build depuis les sources, hot-reload API
#   test - Images DockerHub, Traefik externe HTTPS, domaines de test
#   prod - Images DockerHub, Traefik externe HTTPS, domaines de prod
```

Les services de persistance (`postgres`, `redis`) n'ont pas de profils ils sont donc démarrés dans tous les cas ; les services `dev` (Traefik local, PgAdmin, API buildée depuis les sources, front) et les services `test/prod` (`api_server`, `front_server`, images Docker Hub, TLS Let's Encrypt) sont mutuellement exclusifs. Bénéfice : l'environnement de développement reproduit la topologie de production (reverse proxy, réseau, healthchecks) tout en gardant ses spécificités locales — c'est une application du principe de parité dev/prod.

En dev, l'API monte le code source en volume pour le hot-reload via nodemon, et le front est routé par un Traefik local dont le dashboard (`localhost:8080`) sert d'outil de diagnostic du routage ([docker-compose.yml](#), service `api`, lignes 125–133) :

```
api:
  profiles: ["dev"]
  image: ${IMAGE_API}
  build:
    context: ./my_memo_master_api
    dockerfile: Dockerfile
  volumes:
    - ./my_memo_master_api:/app
    - api-node-modules:/app/node_modules
```

Développement hors Docker : la bascule SQLite / PostgreSQL

Pour permettre un développement encore plus léger (hors docker), l'API peut tourner sur SQLite. La bascule est automatique et repose sur une seule condition — la présence de `PG_HOST` dans l'environnement (`my_memo_master_api/models/index.js`) :

```
const dbmsConfig = require('../config/dbms.config') // PostgreSQL + pool
const dbConfig = require('../config/db.config')      // SQLite fichier local

// Instantiate Sequelize using the right configuration for the current environment
const instance = new Sequelize(process.env.PG_HOST ? dbmsConfig : dbConfig)
```

Cette décision est tracée dans le journal des décisions (`.agents/DECISIONS.md`, « [2026-06-03] SQLite en dev, PostgreSQL en prod ») avec ses conséquences assumées : Sequelize doit rester compatible avec les deux dialectes et les migrations doivent être testées sur les deux. Ce même mécanisme sert le harnais de test (section 4) : les tests s'exécutent sur SQLite in-memory, sans dépendance à une base externe — condition indispensable pour tourner dans les runners CI.

Configuration par variables d'environnement

Toute la configuration passe par des variables d'environnement, jamais par des valeurs codées en dur : le fichier `.env.example` (racine) sert de contrat documenté, et chaque environnement a son `.env` propre non versionné (`.gitignore`). Côté API, les accès sont centralisés dans `my_memo_master_api/config/` (`db.config.js`, `dbms.config.js`, `redis.config.js`, `storage.config.js`, `swagger.config.js`) ; côté front dans `src/config.js` — règle formalisée dans `.agents/CONVENTIONS.md`. L'homogénéité de style entre contributeurs est en outre garantie par un `.editorconfig` à la racine.

1.2 Environnements de déploiements

Le projet est déployé dans trois environnements distincts, alimenté par une branche git dédiée :

Branche git	Environnement	Infrastructure	Images Docker Hub
dev	test	VPS (Docker Compose + Traefik)	mymemomaster_test_api / _front
staging	preprod	Kubernetes Infomaniak mutualisé (Helm)	mymemomaster_preprod_api / _front
main	prod	Kubernetes Infomaniak dédié (Helm)	mymemomaster_api / _front

Cette correspondance est documentée dans le `README.md` (partie 3) et implémentée dans le pipeline `.github/workflows/cd.yml`. Chaque environnement dispose de sa propre architecture :

Test (VPS, Docker Compose). L'environnement de test tourne sur un VPS avec le fichier dédié [server_docker_compose/docker-compose.yml](#), déployé automatiquement par le pipeline CD (section 6). Il comprend PostgreSQL et Redis avec healthchecks, l'API, le front, PgAdmin, et un service `backup` que j'ai écrit pour produire un dump PostgreSQL quotidien avec rétention configurable (extrait des logs du service : `[backup] Service démarré – sauvegarde quotidienne à ${BACKUP_HOUR}h00 UTC, Rétention ${BACKUP_RETENTION_DAYS}j appliquée`).

Preprod et prod (Kubernetes, Helm). Les deux environnements Kubernetes sont déployés par un chart Helm unique ([helm/](#)) : les templates ([helm/templates/](#) — deployments API/front/pgadmin, StatefulSet PostgreSQL, Redis, ConfigMap, Ingress, Services) sont partagés, et chaque environnement n'est qu'un fichier de surcharges ([helm/values-preprod.yaml](#), [helm/values-prod.yaml](#)). Exemple de différenciation assumée en preprod :

```
# Deployment sans PVC – pertes de cache acceptables en preprod
redis:
  persistent: false
```

C'est une décision de conception : la preprod économise un volume persistant là où la prod n'a pas ce droit. De même, PgAdmin est activé en preprod « pour faciliter l'administration » (commentaire du fichier de values) alors que l'exposition d'outils d'administration en prod est restreinte.

Isolation des environnements. Chaque environnement a son espace propre : namespaces Kubernetes distincts (`mymemomaster-preprod` / `mymemomaster`), réseaux Docker nommés par environnement (`my_memo_master_${ENVIRONMENT}_network`), images Docker Hub distinctes (`_test_`, `_preprod_`, sans suffixe pour la prod), secrets gérés hors dépôt (secrets GitHub Actions et Secrets Kubernetes créés manuellement — procédure documentée dans le [README.md](#), partie 3). Le pipeline CD vérifie même que le `.env` présent sur le VPS correspond bien à l'environnement attendu avant de déployer (`grep -q '^ENVIRONMENT=test$' .env`, [.github/workflows/cd.yml](#)).

1.3 Outils de suivi de qualité et de performance

Qualité du code : linters et formateurs

Les deux applications ont chacune leur configuration de lint, exécutée localement et **bloquante en CI** (section 2) :

- **API** : ESLint v9 en configuration *flat* ([my_memo_master_api/eslint.config.mjs](#)) — règles [@eslint/js](#) recommandées + `eslint-config-prettier`, globals Node/Jest déclarés ;
- **Front** : ESLint v8 + `eslint-plugin-vue` + Prettier ([my_memo_master_front/package.json](#), script `lint` couvrant `.vue`, `.js`, `.jsx`, `.cjs`, `.mjs`).

Les scripts npm normalisent l'usage : `npm run lint` et `npm run format` des deux côtés du monorepo.

Qualité fonctionnelle : les harnais de test

Jest + Supertest côté API, Vitest + `@vue/test-utils` côté front, exécutés en CI à chaque push. Le harnais est détaillé en section 4 ; au titre de l'outillage d'environnement, le point notable est qu'il est **auto-suffisant** (SQLite in-memory, mock du modèle d'IA dans [my_memo_master_api/mocks/](#)) : aucun service externe n'est requis pour valider un commit.

Suivi de performance et de disponibilité

- **Healthchecks** à tous les étages : `pg_isready` sur PostgreSQL, `redis-cli ping` sur Redis, `wget --spider` sur le front ([docker-compose.yml](#) ; HEALTHCHECK aussi intégré à l'image front, [my_memo_master_front/Dockerfile](#)). Les dépendances de démarrage s'appuient dessus (`depends_on: condition: service_healthy`), et le pipeline CD s'en sert comme critère de succès du déploiement (section 6).
- **Limites de ressources** : chaque service Docker Compose déclare `limits` et `reservations` CPU/mémoire paramétrables (ex. API : 0.50 CPU / 512 Mo par défaut), et les manifests Helm déclarent `requests/limits` Kubernetes par environnement ([helm/values-preprod.yaml](#)). Le pool de connexions PostgreSQL est dimensionné par configuration ([my_memo_master_api/config/dbms.config.js](#) : `max`, `min`, `acquire`, `idle`).
- **Logs applicatifs structurés** : logger Winston + logs d'accès HTTP Morgan pipés dans Winston, format `combined` en production ([my_memo_master_api/app.js](#), lignes 62–67).
- **Métriques applicatives RED/USE (Prometheus)** : l'API est instrumentée avec `prom-client` — histogramme `http_request_duration_seconds` et compteur `http_requests_total` labellisés méthode/route/code (RED), métriques process Node (CPU, mémoire, event-loop — USE) via `collectDefaultMetrics` ([my_memo_master_api/helpers/metrics.js](#), [middlewares/metrics.middleware.js](#)). L'endpoint `/metrics` est servi par un **serveur HTTP dédié** (port 9090, [server.js](#)) jamais exposé par l'Ingress — décision documentée dans [.agents/DECISIONS.md](#) ; les annotations `prometheus.io/*` sont posées sur les pods ([k8s/](#)). L'instrumentation est testée (9 tests dédiés).
- **Notifications temps réel** : chaque exécution CI/CD notifie un canal Discord (succès/échec, branche concernée) — [.github/workflows/notify_ci.yml](#) et job `notify` de [.github/workflows/cd.yml](#). L'équipe est prévenue d'une régression sans avoir à surveiller GitHub.

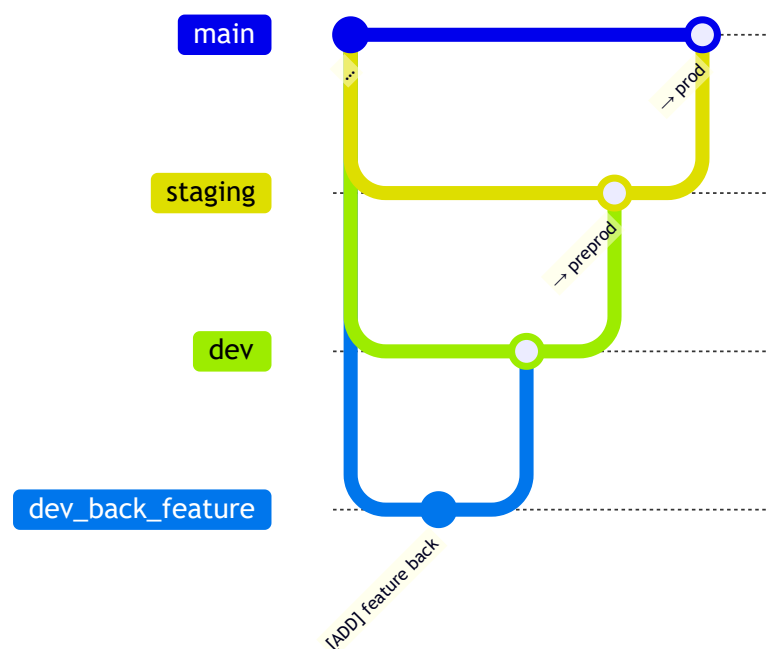
Analyse statique continue : SonarQube (temporairement désactivé)

Le projet a intégré une analyse SonarQube auto-hébergée : la configuration du scanner est versionnée ([sonar-project.properties](#)) et le job d'analyse existe dans le pipeline CI avec une

distinction prod/preprod par token ([.github/workflows/ci.yml](https://github.com/workflows/ci.yml), lignes 68–91). Ce job est **actuellement commenté** : le serveur SonarQube que j'auto-hébergeais est hors service et sa reconfiguration est planifiée. J'assume ce choix de transparence plutôt que de présenter un outil comme actif alors qu'il ne l'est pas ; dans l'intervalle, la couverture qualité reste assurée par le triptyque bloquant lint + tests + build en CI, et la revue de sécurité a été menée manuellement (audit OWASP, section 5.2).

1.4 Dépôt de gestion du code source

Le code source est hébergé sur GitHub ([entrezunfredici/MyMemoMaster](https://github.com/entrezunfredici/MyMemoMaster)), dépôt unique pour l'API, le front, l'infrastructure et la documentation. Ce choix du monorepo est une décision de conception : les deux applications, l'infrastructure et la documentation évoluent dans un même historique git, ce qui permet à une même pull request de porter une fonctionnalité de bout en bout (migration de base, endpoint, écran, test, manifeste de déploiement). J'ai défini la stratégie de branches suivante, documentée dans le [README.md](#) (« Méthode de travail ») et **outillée par le pipeline CI** (la matrice de jobs cible l'API ou le front selon le préfixe de la branche — section 2) :



- les branches de travail sont préfixées `dev_back_*` ou `dev_front_*` selon le périmètre ;
- `dev` intègre en continu et alimente l'environnement de **test** ;
- `staging` promeut vers la **preprod** ; `main` vers la **prod** ;
- les commits suivent une convention de préfixes `[ADD]` / `[IMP]` / `[REF]` / `[FIX]`, ce qui rend l'historique exploitable comme journal des évolutions et des correctifs (utilisé en sections 8 et 9).

La méthode de travail impose l'ordre « tests unitaires → code → documentation Swagger » pour chaque fonctionnalité ([README.md](#), étape 3) : l'environnement n'est pas qu'un outillage, il porte aussi le processus de développement.

Section 2 — Intégration continue

Compétence couverte : C2.1.2 — Configurer le système d'intégration continue dans le cycle de développement du logiciel en fusionnant les codes sources et en testant régulièrement les blocs de code afin d'assurer un développement efficient qui réduit les risques de régression.

2.1 Protocole d'intégration continue

J'ai implémenté l'intégration continue avec GitHub Actions dans le workflow [.github/workflows/ci.yml](#). Le protocole tient en une phrase : **aucun code ne progresse vers un environnement sans avoir passé tests, lint et build sur un runner neutre**. Le workflow se déclenche à chaque push sur les branches d'intégration (**main**, **staging**, **dev**) et sur les branches de travail (**dev_back_***, **dev_front_***, ***devops***) :

```
on:
  push:
    branches:
      - main
      - staging
      - dev
      - '*devops*'
      - 'dev_back_*'
      - 'dev_front_*
```

Le déclenchement sur les branches de travail — et pas seulement sur les branches d'intégration — est un choix délibéré : le développeur reçoit le verdict de la CI **avant** de fusionner dans **dev**, ce qui déplace la détection des régressions au plus tôt dans le cycle (principe *shift-left*). La fusion dans une branche d'intégration re-déclenche une validation complète, qui conditionne ensuite le déploiement (section 2.3).

2.2 Une matrice de jobs adaptée au monorepo

Un monorepo pose un problème classique de CI : re-valider systématiquement les deux applications gaspille du temps de runner quand une branche ne touche qu'un côté. J'ai résolu ce point avec un job **setup** qui construit **dynamiquement la matrice de jobs selon le préfixe de la branche** ([.github/workflows/ci.yml](#), job **setup**) :

```
- id: set-matrix
  run: |
    BRANCH="${GITHUB_REF##*/}"
    if [[ "$BRANCH" == dev_front_* ]]; then
      echo 'matrix={"include":[{"service":"front","path":"my_memo_master_front"}]}' >>
$GITHUB_OUTPUT
    elif [[ "$BRANCH" == dev_back_* ]]; then
      echo 'matrix={"include":[{"service":"api","path":"my_memo_master_api"}]}' >>
$GITHUB_OUTPUT
    else
      echo 'matrix={"include":[{"service":"api","path":"my_memo_master_api"},
```

```
{ "service": "front", "path": "my_memo_master_front" } ] } ' >> $GITHUB_OUTPUT  
fi
```

Une branche `dev_front_*` ne valide que le front, une branche `dev_back_*` que l'API ; les branches d'intégration (`dev`, `staging`, `main`) valident **toujours les deux**, car c'est là que les contributions front et back se rencontrent — c'est précisément le moment où une régression d'interface entre les deux peut apparaître. La stratégie de nommage des branches (section 1.4) n'est donc pas cosmétique : elle pilote l'outillage.

2.3 Séquence de validation d'un bloc de code

Chaque entrée de la matrice exécute la même séquence, définie dans le job `test_and_lint` :

Étape	Commande	Rôle
1. Installation reproductible	<code>npm ci</code>	installe exactement les versions verrouillées dans <code>package-lock.json</code> — pas de dérive de dépendances entre le poste du développeur et le runner
2. Tests	<code>npm run test</code>	Jest + Supertest (API) / Vitest (front) — détection des régressions fonctionnelles
3. Lint	<code>npm run lint</code>	ESLint — détection des défauts de qualité et erreurs statiques
4. Audit des dépendances	<code>npm audit -- omit=dev -- audit-level=high</code>	bloque si une dépendance de production porte une vulnérabilité high/critical connue (OWASP A06) — les devDependencies, absentes des images déployées, sont exclues
5. Build (front uniquement)	<code>npm run build</code>	vérifie que le bundle Vite de production se construit

Trois réglages du workflow méritent justification :

- `node-version: 22` avec cache npm par `package-lock.json` : le runner utilise **la même version de Node que les images Docker de production** (`node:22-bookworm-slim` / `node:22-alpine`, cf. Dockerfiles section 6.3) — on teste ce qu'on déploie ;
- `fail-fast: false` sur la matrice : si l'API échoue, la validation du front va quand même au bout — le développeur reçoit le diagnostic complet en une passe au lieu de découvrir les échecs un par un ;
- `permissions: contents: read` : le workflow ne dispose que du droit de lecture du dépôt, application du principe du moindre privilège aux jetons GitHub Actions.

Le harnais est exécutable en CI sans aucun service externe : SQLite in-memory pour la base, mock du modèle d'IA (`moduleNameMapper` vers `my_memo_master_api/mocks/@xenova/transformers.js`,

déclaré dans [my_memo_master_api/package.json](#)). C'est ce qui rend la boucle rapide et déterministe.

2.4 La CI comme porte d'entrée du déploiement

Le pipeline de déploiement ([.github/workflows/cd.yml](#)) ne se déclenche pas sur push : il s'abonne à la fin du workflow CI et vérifie sa conclusion :

```
on:
  workflow_run:
    workflows:
      - CI
    types:
      - completed
    branches: [main, staging, dev]
```

```
push_images:
  if: >
    github.event.workflow_run.conclusion == 'success' &&
    github.event.workflow_run.event == 'push'
```

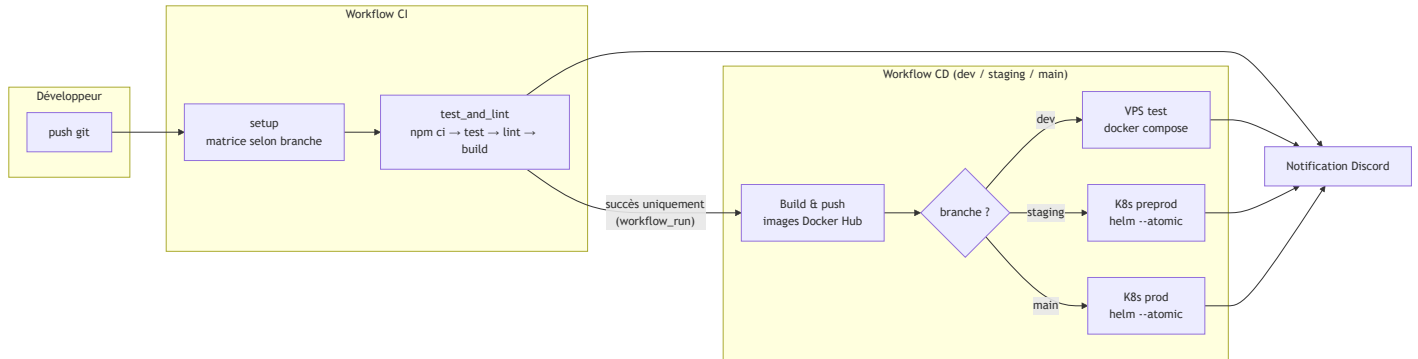
Cette architecture en deux workflows chaînés garantit structurellement qu'**aucune image Docker n'est construite ni déployée à partir d'un commit dont les tests ou le lint ont échoué**. C'est le cœur du protocole d'intégration continue : la séquence d'intégration (fusion → validation → livraison) est outillée, pas seulement conventionnelle.

2.5 Boucle de retour vers l'équipe

Chaque exécution CI notifie un canal Discord via le workflow dédié [.github/workflows/notify_ci.yml](#) : message de succès, ou message d'échec indiquant la branche fautive ("Tests failed on branch **\$BRANCH_NAME!**"). Le pipeline CD envoie de son côté un bilan de déploiement (✅/❌ **Déploiement **<branche>** réussi/échoué**, job **notify** de [cd.yml](#)). Une régression introduite sur **dev** est ainsi connue de l'équipe en quelques minutes, sans surveillance active de l'interface GitHub.

[SCREENSHOT ICI : canal Discord montrant une notification d'échec CI et une notification de déploiement réussi]

2.6 Vue d'ensemble du pipeline



Légende : rectangles = jobs GitHub Actions ; losange = aiguillage par branche ; la flèche CI → CD n'existe que si la CI conclut en succès.

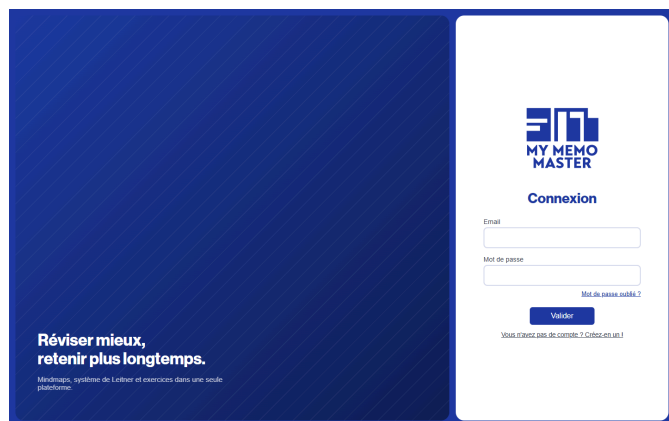
Section 3 — Prototypage et conception de l'application

Compétence couverte : C2.2.1 — Concevoir un prototype de l'application logicielle en tenant compte des spécificités ergonomiques et des équipements ciblés (ex : web, mobile...) afin de répondre aux fonctionnalités attendues et aux exigences en termes de sécurité.

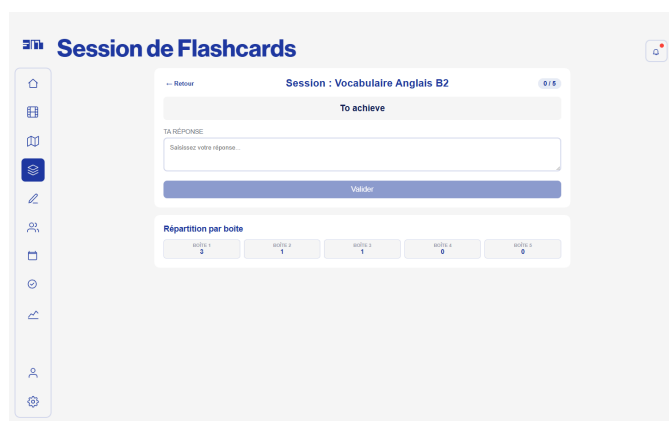
3.1 Démarche de prototypage : de la maquette au composant

La conception des interfaces suit trois niveaux de raffinement successifs :

Niveau 1 — Prototype interactif versionné. Les écrans principaux ont été prototypés en amont sous forme d'une maquette **interactive et navigable**, réalisée avec l'outil de design de Claude (assistance IA — démarche assumée et cohérente avec l'outillage du projet), exportée en **HTML autonome** et versionnée dans le dépôt : [docs/prototype/MyMemoMaster - Standalone.html](https://github.com/MyMemoMaster/MyMemoMaster-standalone.html) (14 écrans : authentification, accueil, mindmaps, flashcards et session Leitner, exercices, classe, calendrier, to-do, KPI, profil, réglages — procédure d'ouverture dans [docs/prototype/README.md](https://github.com/MyMemoMaster/MyMemoMaster-standalone.html)). Le prototype fixe l'identité visuelle et les parcours ; les captures ci-dessous sont **générées automatiquement depuis ce fichier** par script Puppeteer ([docs/prototype/captures/](https://github.com/MyMemoMaster/MyMemoMaster-standalone.html)), donc reproductibles :

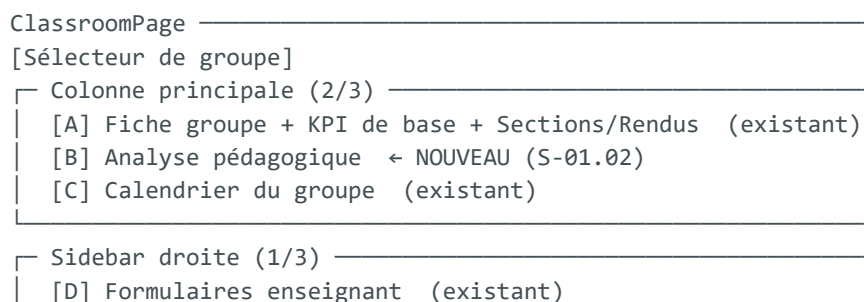


Connexion



Session de révision (répartition par boîte)

Des maquettes graphiques Figma antérieures existent également, externes au dépôt ; le prototype HTML versionné en constitue la trace exploitable et vérifiable. La galerie complète des 16 écrans capturés figure en **Annexe A**.



Versionner ces maquettes avec le code est une décision de méthode : la spécification d'interface évolue dans les mêmes pull requests que son implémentation, et le document indique explicitement

ce qui est existant et ce qui est nouveau — le prototype s'inscrit dans un incrément, pas dans une refonte.

Niveau 3 — Implémentation Vue. Chaque maquette se concrétise en une page (`src/pages/[Name]Page.vue`) composée de composants réutilisables (`src/components/[Name]Component.vue`), conventions formalisées dans [.agents/CONVENTIONS.md](#).

3.2 Un prototype fonctionnel couvrant les fonctionnalités attendues

Le « prototype » au sens du référentiel est ici l'application elle-même, fonctionnelle et manipulable en autonomie : elle est déployée sur les trois environnements et utilisable de bout en bout (inscription, vérification email, connexion, création et révision de contenus). La couverture des fonctionnalités principales se lit directement dans l'arborescence des 28 pages de [my_memo_master_front/src/pages/](#) :

Fonctionnalité attendue	Pages implémentées
Systèmes de Leitner (flashcards)	<code>FlashcardsPage</code> , <code>FlashcardsCardsPage</code> , <code>FlashcardsSessionPage</code>
Cartes mentales	<code>MindmapsPage</code> (+ composants dédiés <code>MindmapComponent</code> , <code>components/mindmap/</code>)
Exercices	<code>ExercisesPage</code> , <code>ExerciseDetailPage</code> , <code>CreateTestPage</code>
Organisation	<code>CalendarPage</code> , <code>TodoPage</code> , <code>SubjectsPage</code>
KPI	<code>KpiPage</code>
Groupes classes / établissements	<code>ClassroomPage</code> + vues par rôle (<code>ClassroomEnseignantView</code> , <code>ClassroomEtudiantView</code> , <code>ClassroomEtablissementView</code> , <code>ClassroomPlateformeView</code>)
Compte et cycle de vie utilisateur	<code>login/</code> , <code>register/</code> , <code>VerifyEmailPage</code> , <code>ForgotPasswordPage</code> , <code>ResetPasswordPage</code> , <code>ProfilePage</code> , <code>AccountPage</code> , <code>SettingsPage</code>
Prise en main	<code>OnboardingPage</code> , <code>TutorialsPage</code>

Les composants d'interface génériques exigés par le critère (fenêtres, boutons, menus...) existent en tant que composants réutilisables : `ButtonComponent`, `ModalComponent`, `DropdownComponent`, `MenuItemComponent`, `LoaderComponent`, `PillComponent`, `ToggleButton`, etc. ([my_memo_master_front/src/components/](#)).

[SCREENSHOT ICI : page `FlashcardsSessionPage` en cours de session de révision, montrant les composants réels (boutons, modale, indicateur de boîte Leitner)]

3.3 Spécificités ergonomiques et équipements ciblés

L'équipement ciblé est le **navigateur web, desktop et mobile**. Trois dispositions concrètes en découlent :

- **Responsive design** : l'interface est construite avec Tailwind CSS et ses classes de points de rupture (`sm:`, `md:`, `lg:`) ; les maquettes de [diagrams/dashboard_enseignant_ui.md](#) spécifient d'ailleurs des colonnes 2/3 – 1/3 qui se replient sur mobile.
- **PWA (Progressive Web App)** : le front est installable sur mobile via le plugin `vite-plugin-pwa` configuré dans [my_memo_master_front/vite.config.js](#) — manifeste complet (nom, icônes 192/512, couleurs de thème) et mise à jour automatique du service worker :

```
VitePWA({
  registerType: 'autoUpdate',
  workbox: {
    // Aucun asset pré-caché : évite le cache multi-Mo par défaut.
    // Les assets sont toujours servis depuis le réseau.
    globPatterns: [],
    cleanupOutdatedCaches: true,
  },
  manifest: {
    name: env.VITE_APP_NAME,
    ...
```

Le choix de ne **pas** pré-cacher les assets est une décision de conception commentée dans le fichier : sur une application dont le contenu est essentiellement dynamique (données de révision de l'utilisateur), un pré-cache Workbox de plusieurs Mo dégraderait la première installation sans bénéfice réel.

- **Guidage utilisateur** : l'ergonomie de prise en main est traitée comme une fonctionnalité à part entière — parcours d'onboarding persisté côté serveur (module `OnboardingState` de l'API), page de tutoriels, notifications toast (`vue-toastification`), états vides explicites (`NoItemComponent`), indicateurs de chargement (`LoaderComponent`), titres d'onglet dynamiques par page (`document.title` dans le guard de navigation, [src/router/index.js](#)).

Une règle d'ergonomie issue d'un défaut réellement rencontré est même codifiée dans les conventions du projet ([.agents/CONVENTIONS.md](#)) : les popups et panneaux flottants doivent déclarer un fond opaque explicite pour rester lisibles quelle que soit la page en dessous.

3.4 Exigences de sécurité intégrées au prototype

La sécurité n'est pas reportée à la couche API : le prototype front l'intègre dès la navigation.

Contrôle d'accès à la navigation. Le routeur applique deux guards à chaque changement de route ([my_memo_master_front/src/router/index.js](#)) :

```
// Guard authentication
if (to.meta.private === true) {
  if (!authStore.authenticated || !authStore.token) {
    authStore.logout(false, null)
    return next({ path: '/auth' })
  }
}

// Guard rôles : meta.roles = [1, 4] signifie "admin plateforme ou admin établissement seulement"
if (to.meta.roles && to.meta.roles.length > 0) {
  const userId = authStore.user?.roleId ?? null
  if (!to.meta.roles.includes(userId)) {
    return next({ path: '/' })
  }
}
```

Les routes déclarent leur exigence dans leurs métadonnées ([meta.private](#), [meta.roles](#)) : ajouter un écran protégé ne demande aucune logique nouvelle, seulement une déclaration — le contrôle d'accès est systématique par construction. Ce contrôle côté client est un confort d'expérience utilisateur (redirection immédiate) ; l'autorité reste côté API, où chaque route privée est protégée par middleware JWT et RBAC (section 5.2).

Gestion de session sécurisée. Le client HTTP centralisé

([my_memo_master_front/src/helpers/api.js](#)) implémente le renouvellement silencieux de session : sur une réponse 401, un intercepteur tente d'échanger le refresh token puis rejoue la requête une seule fois (drapeau [_retried](#) contre les boucles infinies) ; si le renouvellement échoue, l'utilisateur est déconnecté proprement :

```
axiosApi.interceptors.response.use(async (response) => {
  if (response.status === 401 && !response.config._retried) {
    const refreshed = await _tryRefreshToken()
    if (refreshed) {
      response.config._retried = true
      removeHeader(response.config.headers, 'Authorization')
      return axiosApi(response.config)
    }
  }
  return response
})
```

Ce mécanisme est la contrepartie ergonomique d'une exigence de sécurité : c'est lui qui rend acceptable un jeton d'accès à durée courte (15 minutes, décision documentée dans [.agents/DECISIONS.md](#) et [docs/SECURITY_AUDIT_OWASP.md](#)) sans dégrader l'expérience — l'utilisateur ne se voit jamais demander de se reconnecter toutes les 15 minutes. Le front intègre aussi un indicateur de robustesse du mot de passe à l'inscription ([PasswordStrengthComponent.vue](#)).

Section 4 — Harnais de tests unitaires

Compétence couverte : C2.2.2 — Développer un harnais de test unitaire en tenant compte des fonctionnalités demandées afin de prévenir les régressions et de s'assurer du bon fonctionnement du logiciel.

4.1 État du harnais : les chiffres

Les chiffres ci-dessous sont issus d'une exécution réelle des deux suites sur le code actuel du dépôt (et non d'une déclaration) :

Harnais	Outillage	Volume	Résultat	Durée
API	Jest + Supertest	80 suites, 1 447 tests	1 447 passés, 0 échec	~40 s
Front	Vitest + @vue/test-utils + @pinia/testing	37 fichiers, 562 tests	562 passés, 0 échec	~35 s

Répartition par couche, révélatrice de la stratégie de test :

Couche testée	Fichiers	Tests	Ce qui est validé
test/controllers/ (API)	34	791	contrat HTTP de chaque endpoint : codes de statut, corps de réponse, cas d'erreur
test/services/ (API)	34	554	logique métier isolée : algorithmes, règles de droits, cas limites
test/middlewares/ (API)	4	31	JWT, RBAC, rate limiting, instrumentation métriques (tests de sécurité dédiés)
test/models/ (API)	2	18	contraintes de modèles sensibles (AuditLog, Etablissement)
test/helpers/ (API)	2	14	helpers transverses : métriques Prometheus, signatures binaires des uploads (magic bytes)
test/bdd/ (API)	4	39	scénarios fonctionnels de bout en bout (sessions complètes)
test/components/ (front)	17	256	rendu et interactions des pages/composants critiques
test/stores/ (front)	14	228	state management Pinia : actions, mutations d'état, appels API mockés
test/router/, composables/, helpers/	4	73	guards de navigation, logique réutilisable

Couche testée	Fichiers	Tests	Ce qui est validé
(front)			
test/a11y/ (front)	1	4	accessibilité runtime (axe-core) des composants critiques — non-régression RGAA

La pyramide est assumée : la masse des tests porte sur les couches controller et service de l'API — là où vivent le contrat public et la logique métier — complétée par des tests fonctionnels transverses ([test/bdd/](#)) qui valident les enchaînements réels.

4.2 Architecture du harnais API : hermétique et déterministe

Un harnais n'est utile contre les régressions que s'il est **rapide, isolé et reproductible** — sinon il n'est pas exécuté. J'ai conçu celui de l'API pour tourner sans aucun service externe. La configuration Jest tient dans [my_memo_master_api/package.json](#) :

```
"jest": {
  "testMatch": [ "**/test/**/*.spec|test).[tj]s?(x)" ],
  "moduleNameMapper": {
    "^@xenova/transformers$": "<rootDir>/__mocks__/@xenova/transformers.js"
  },
  "testTimeout": 60000,
  "forceExit": true,
  "setupFiles": [ "<rootDir>/test/setup.js" ]
}
```

Trois mécanismes d'isolation, visibles dans l'en-tête du test de session Leitner ([my_memo_master_api/test/bdd/leitner.session.test.js](#)) :

```
// Base SQLite en mémoire – seul Semantic.service est mocké (dépendance NLP externe ~30s).
// Les autres couches (controller → service → model → DB) sont réelles.
process.env.DB_STORAGE = ':memory:'

jest.mock('../services/Semantic.service', () => ({ gradeSemantic: jest.fn() }))
jest.mock('../jobs/fifo.cron', () => ({ startFifoCron: jest.fn() }))
jest.mock('../jobs/reminder.worker', () => ({ startReminderWorker: jest.fn() })))
```

- **Base de données** : SQLite in-memory, une instance par fichier de test — aucune pollution entre tests, aucun PostgreSQL requis (c'est la contrepartie directe de la bascule de dialecte décrite en section 1.1) ;
- **Modèle d'IA** : le pipeline [@xenova/transformers](#) (~30 s de chargement) est remplacé par un mock global ([my_memo_master_api/mocks/@xenova/transformers.js](#)) — sauf dans les tests dédiés au service sémantique qui contrôlent leur propre mock ;

- **Jobs asynchrones** : crons et worker BullMQ sont neutralisés pour ne pas dépendre de Redis ni introduire de timing non déterministe.

Le fichier [test/setup.js](#) désactive par défaut le rate limiting (`RATE_LIMIT_DISABLED = 'true'`), **sauf** pour les tests de sécurité qui le réactivent afin de valider précisément ce comportement ([test/middlewares/security.test.js](#)) — l'exception est documentée en commentaire dans le fichier même.

4.3 Jeu de tests couvrant une fonctionnalité : l'algorithme de Leitner

Pour illustrer la couverture d'une fonctionnalité complète, je prends le cœur métier de l'application : la répétition espacée. Elle est testée à deux niveaux complémentaires.

Niveau unitaire — [test/services/LeitnerCard.service.test.js](#) (21 tests). Les intitulés suivent systématiquement le schéma *méthode* – *condition* – *comportement attendu*, et couvrent le cas nominal, les cas limites et les erreurs attendues :

```
correctResponse - bonne réponse - avance à la boîte suivante et incrémente correct_count
correctResponse - mauvaise réponse - retour boîte 1 et incrémente incorrect_count
correctResponse - bonne réponse en boîte 5 - reste en boîte 5 (plafonnement)
correctResponse - carte introuvable - retourne null
correctResponse - aucune réponse correcte en base - retourne null sans grading
addCard - droits insuffisants - lève une erreur
resolveUserRights - utilisateur partagé avec écriture - canAdd et canEdit true
resolveUserRights - utilisateur sans accès - aucun droit
```

On y lit l'algorithme (progression, rétrogradation, plafonnement en boîte 5) **et** le modèle de droits (propriétaire / partagé en écriture / sans accès) — les deux dimensions de la fonctionnalité.

Niveau fonctionnel — [test/bdd/leitner.session.test.js](#) (10 scénarios). Une session réelle est déroulée via Supertest à travers toutes les couches (routes → controllers → services → modèles → base) : carte jamais révisée retournée comme due, avancement en boîte 2 après bonne réponse, réapparition après dépassement de `next_review_at`, et les erreurs de contrat (401 sans token, 404 carte inexistante, 400 corps invalide). Ce niveau attrape les régressions d'intégration que les tests unitaires par couche ne voient pas.

4.4 Harnais front : tester le comportement, pas l'implémentation

Côté front, les tests montent réellement les composants (`mount` de `@vue/test-utils`) avec un store Pinia de test (`createTestingPinia`) et des dépendances stubbing explicites. Exemple des 13 cas de [test/components/FlashcardsSessionPage.test.js](#), qui valident le parcours utilisateur complet d'une session de révision :

affiche "Chargement" pendant onMounted, puis le masque après résolution
affiche "Aucune carte à réviser" quand dueCards est vide
le bouton Valider est désactivé quand la réponse est vide
bonne réponse – affiche le feedback vert avec le score
mauvaise réponse – affiche le feedback rouge avec la correction attendue
affiche "Session terminée" après la dernière carte
bouton Retour + confirmation – redirige vers /flashcards

Ces intitulés décrivent des comportements observables par l'utilisateur (états affichés, boutons actifs, redirections) et non des détails d'implémentation : un refactoring interne du composant ne casse pas ces tests, une régression fonctionnelle les casse — c'est exactement le rôle d'un harnais anti-régression.

4.5 Couverture et exécution systématique

La couverture de code de l'API, mesurée par `npx jest --coverage` sur le code actuel du dépôt, s'établit à :

Métrique	Couverture
Lignes	86,17 %
Instructions	85,74 %
Fonctions	84,81 %
Branches	66,44 %

La majorité du code développé est donc couverte, conformément au critère du référentiel. La couverture de branches, plus basse, est typique d'un code défensif (branches d'erreur rares, garde-fous) ; les branches critiques — algorithme Leitner, droits, authentification — sont, elles, couvertes explicitement par les cas listés en 4.3.

Le harnais entier — les deux suites — est exécuté **à chaque push** par la CI et conditionne tout déploiement (sections 2.3 et 2.4). La méthode de travail du projet place par ailleurs l'écriture des tests **avant** le code dans le cycle de développement d'une fonctionnalité ([README.md](#), étape 3 : « 1. tests unitaires, 2. code, 3. documentation swagger »).

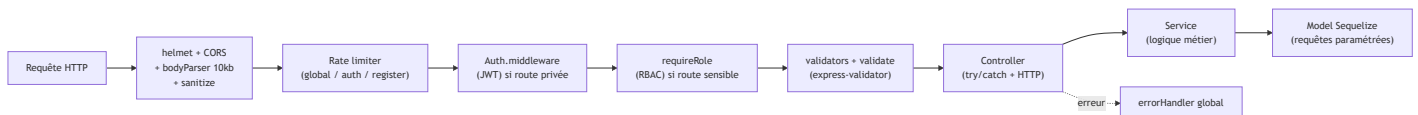
Section 5 — Développement : évolutivité, sécurisation, accessibilité

Compétence couverte : C2.2.3 — Développer le logiciel en veillant à l'évolutivité et à la sécurisation du code source, aux exigences d'accessibilité et aux spécifications techniques et fonctionnelles définies, pour garantir une exécution conforme aux exigences du client.

5.1 Une architecture logicielle structurée pour la maintenabilité

Le pipeline d'une requête

J'ai imposé sur toute l'API une architecture en couches stricte — **route** → **middlewares** → **controller** → **service** → **model** — formalisée comme règle de projet dans [.agents/CONVENTIONS.md](#) (« Pas de logique métier dans les controllers — tout passe par les services »). Chaque requête traverse le même pipeline :



Légende : chaque bloc est un fichier ou dossier réel de [my_memo_master_api/](#) ; les flèches pleines suivent le traitement nominal, la flèche pointillée le chemin d'erreur.

La composition est lisible directement dans la déclaration des routes ([my_memo_master_api/routes/User.routes.js](#)) :

```
router.post('/register', registerLimiter, userValidators.register, validate, user.register)
router.post('/login', authLimiter, userValidators.login, validate, user.login)
router.put('/:id', authMiddleware, userValidators.update, validate, user.update)
```

Des responsabilités étanches

Le controller ne fait que du HTTP. Son rôle se limite à try/catch, appel du service et traduction en code de statut, messages en français ([my_memo_master_api/controllers/Subject.controller.js](#)) :

```
exports.findAll = async (req, res) => {
  try {
    const data = await subjectService.findAll()
    res.status(200).send(data)
  } catch (error) {
    logger.error(error?.message || error)
    res.status(500).send({ message: "Une erreur s'est produite lors de la récupération des sujets." })
  }
}
```

La validation est déclarative et centralisée. Chaque entité a son fichier de règles ([my_memo_master_api/validators/User.validators.js](#) — longueur et complexité du mot de passe, normalisation d'email...), appliquées par un middleware unique [validate.middleware.js](#) qui renvoie un 400 uniforme. **La gestion d'erreur est un filet global** ([middlewares/errorHandler.middleware.js](#)) : toute erreur non gérée est loggée puis traduite en réponse JSON — en production le message

interne est masqué (`isProd ? 'Erreur interne du serveur.' : err.message`), ce qui évite de divulguer des détails d'implémentation (voir 5.2).

Cette uniformité (36 controllers, 37 services, 34 fichiers de validators construits sur le même patron, conventions de nommage documentées) est ce qui rend le code **maintenable par quelqu'un d'autre que son auteur** : localiser une règle métier, une règle de validation ou un contrat HTTP ne demande aucune connaissance tribale, seulement la convention.

5.2 Sécurisation du code source : démarche OWASP Top 10

La démarche : un audit formalisé, versionné, suivi

Plutôt que d'égrener des mesures de sécurité au fil de l'eau, j'ai conduit un **audit complet contre l'OWASP Top 10**, dont le livrable est versionné dans le dépôt : [docs/SECURITY_AUDIT_OWASP.md](#) (ticket M-00b.07). Ce document qualifie chaque constat (catégorie OWASP, sévérité, statut) et distingue trois ensembles : vulnérabilités corrigées immédiatement, vulnérabilités hautes corrigées dans le ticket suivant (M-00b.07b), et risques résiduels acceptés et documentés. Extrait du tableau de synthèse :

ID	OWASP	Titre	Sévérité	Statut
F-01/02/03	A01	Routes POST/PUT/DELETE de Fields/Tutorials/Test sans authMiddleware	Critique	✓ Corrigé
F-04	A01	Login sans vérification hasValidatedEmail	Moyenne	✓ Corrigé
F-05/06	A04	Énumération d'emails via forgotPassword / verifyEmail	Moyenne	✓ Corrigé
F-07	A02	generateToken.js utilisait Math.random() (non-crypto)	Faible	✓ Corrigé
F-08	A05	Swagger UI accessible en production	Moyenne	✓ Corrigé

Le backlog de l'audit a été **résorbé par lots successifs**, tracés dans le document : après M-00b.07b, la passe du 2026-07-06 a corrigé les priorités moyennes restantes — CSP explicite (A05-M4), vérification des uploads par magic bytes (A08-M2), journalisation des échecs d'authentification et des refus d'accès (A09-M3, F-M8), doublon d'email en 400 au lieu de 500 y compris sur la race condition de contrainte UNIQUE (F-M4), caractère spécial exigé dans les mots de passe (F-M2), neutralisation de l'injection de logs (F-M7). L'audit assume ce qui **reste** : A07-M1 (pas de révocation JWT — palliatif documenté : expiration 15 min + rotation du refresh token). Présenter au jury les risques résiduels documentés fait partie de la démarche : la sécurité est un processus tracé, pas un état déclaré.

La défense en profondeur implémentée

Chaque étage du pipeline de la section 5.1 porte une mesure, rattachable à une catégorie OWASP :

- **A01 — Contrôle d'accès** : middleware JWT systématique sur les routes privées ([Auth.middleware.js](#)) ; RBAC par [requireRole.middleware.js](#) qui revérifie le rôle **en base à chaque requête** — décision documentée dans [.agents/DECISIONS.md](#) : un rôle révoqué prend effet immédiatement, au prix d'une requête DB, plutôt qu'à l'expiration du JWT ;
- **A02 — Cryptographie** : mots de passe bcrypt ; access token 15 min ; refresh token opaque avec **rotation à chaque renouvellement** ; token de reset password hashé SHA-256 en base, token brut envoyé par email uniquement (décisions des 2026-06-14/15, [.agents/DECISIONS.md](#)) ;
- **A03 — Injection** : requêtes exclusivement via l'ORM Sequelize (paramétrées) ; nettoyage des balises HTML de tous les champs du body par [sanitize.middleware.js](#) ; validation d'entrée déclarative par entité ;
- **A04 — Conception** : anti-énumération d'emails (réponses génériques) ; rate limiting à trois étages ([rateLimit.middleware.js](#)) : global 500 req/15 min, login 10 tentatives **échouées**/15 min ([skipSuccessfulRequests: true](#)), inscription 10/h. Le limiteur global cle-vère par userId extrait du JWT plutôt que par IP — le commentaire du code justifie ce choix : « le keying par userId évite le problème NAT scolaire (plusieurs élèves derrière la même IP) », cas d'usage central pour une application utilisée en classe ;
- **A05 — Configuration** : en-têtes durcis par [helmet](#) avec **CSP explicite** (défauts stricts + [blob:](#) pour les aperçus d'images — auditable dans le code, [app.js](#)) ; CORS en liste blanche par fonction (pas de reflet d'origine inconnue, [app.js](#) lignes 72–93) ; Swagger désactivé en production ; body JSON limité à 10 ko ; [trust proxy](#) réglé pour que le rate limiting voie la vraie IP derrière Traefik ; secrets hors dépôt (section 1.2) ; HTTPS forcé + HSTS par labels Traefik ([docker-compose.yml](#)) ;
- **A06 — Composants vulnérables** : audit npm **bloquant en CI** sur les dépendances de production ([npm audit --omit=dev --audit-level=high](#), section 2.3) — le driver SQLite (dev/test uniquement) a été déplacé en devDependencies pour sortir sa chaîne de build vulnérable des images déployées : **0 vulnérabilité high/critical** sur les deux applications à la date du dossier ;
- **A08 — Intégrité des données** : uploads vérifiés en **deux lignes de défense** ([helpers/fileSignature.js](#)) — croisement extension ↔ MIME déclaré au filtrage, puis **magic bytes lus sur le flux** avant écriture S3 (remplace [AUTO_CONTENT_TYPE](#) qui détectait sans jamais rejeter) : un exécutable renommé en [.png](#) est refusé même avec un MIME forgé ;
- **A09 — Journalisation** : Winston + Morgan (section 1.3), messages d'erreur internes masqués en production, échecs d'authentification et refus d'accès (401/403) journalisés en [warn](#) avec IP, messages d'erreur nettoyés des caractères de contrôle avant écriture (anti log-injection).

Ces mesures sont **elles-mêmes testées** : [test/middlewares/security.test.js](#) et [test/middlewares/Auth.middleware.test.js](#) valident JWT invalide/expiré, RBAC (dont la journalisation des refus) et rate limiting (31 tests, section 4.1) ; la vérification des signatures binaires d'upload a ses 12 tests dédiés ([test/helpers/fileSignature.test.js](#)).

5.3 Accessibilité : audit et plan de mise en conformité

Choix du référentiel

Je retiens le **RGAA 4** (Référentiel Général d'Amélioration de l'Accessibilité) comme référentiel cible : c'est le standard officiel français, fondé sur WCAG 2.1, et le public visé (établissements scolaires, potentiellement soumis à obligation légale d'accessibilité) rend ce choix cohérent avec le produit. Pour un MVP, je l'applique en priorisant les critères à plus fort impact utilisateur (navigation clavier, alternatives textuelles, formulaires, structure), démarche d'échantillonnage inspirée de la checklist OPQUAST.

Résultats de l'audit du code front

J'ai audité le code de `my_memo_master_front/src/` critère par critère. Les décomptes proviennent d'une analyse statique réelle des `.vue` du dépôt.

Points conformes :

Critère (RGAA)	Constat dans le code
Éléments interactifs natifs (7.x, 12.x)	208 <code><button></code> contre 21 <code><div @click></code> : l'écrasante majorité des actions utilise l'élément sémantique, focusable et activable au clavier par défaut
Images (1.x)	Les 5 <code></code> du code portent tous un attribut <code>alt</code> ; les icônes émoji décoratives sont neutralisées (<code>aria-hidden="true"</code> , <code>KpiAlertWidgetComponent.vue</code>)
Fenêtres modales (7.x)	<code>ModalComponent.vue</code> déclare <code>role="dialog"</code> et <code>aria-modal="true"</code> , fermeture au clic hors panneau
Navigation clavier (12.x)	Composants d'édition pilotables au clavier : <code>TagSelectorComponent.vue</code> gère Entrée, Échap, flèches haut/bas et Backspace (<code>@keydown.down.prevent="moveFocus(1)"...</code>) avec <code>aria-label</code> dynamique (<code>Retirer \${tag.name}</code>) ; édition des nœuds de carte mentale via Entrée/Échap
Structuration (9.x)	Hierarchie de titres présente (9 <code><h1></code> , 51 <code><h2></code> , 27 <code><h3></code>) ; 84 <code><label></code> dans les formulaires
Lisibilité	Titres d'onglet par page (<code>document.title</code> , section 3.3) ; états de chargement et états vides explicites

Non-conformités relevées (constats de mon audit, à corriger) :

Critère (RGAA)	Constat	Correction préconisée
8.3 — Langue de la page	index.html déclarait <code><html lang="en"></code> alors que l'application est en français : les lecteurs d'écran prononçaient le contenu avec une voix anglaise	✅ Corrigé pendant la rédaction de ce dossier : <code>lang="fr"</code>
11.1 — Étiquettes de champs	106 <code><input></code> pour 84 <code><label></code> dont 20 seulement avec <code>for=</code> explicite : une partie des champs n'a vraisemblablement pas de nom accessible (les labels enveloppants sont valides, mais l'écart doit être résorbé champ par champ)	✅ Corrigé (campagne outillée 2026-07-06) : l'audit statique a dénombré précisément 111 champs sans nom accessible ; <code>aria-label</code> ajouté sur chacun, ré-audit à 0 — voir docs/AUDIT_RGAA.md
7.x — Gestion du focus	ModalComponent.vue se fermait à Échap mais ne piégeait pas le focus (Tab sortait de la modale) et ne le restituait pas à la fermeture ; le bouton × n'avait pas de nom accessible ; 21 <code><div @click></code> restent inaccessibles au clavier	✅ Corrigé pour la modale pendant la rédaction : focus trap Tab/Shift+Tab, focus initial, restitution à la fermeture, <code>aria-label="Fermer"</code> — couvert par 10 tests (test/components/ModalComponent.test.js). ✅ Les éléments cliquables restants sont tous traités (2026-07-06) : conversion sémantique ou pattern <code>ARIA role="button"/tabindex/@keydown</code> , motifs d'overlay documentés comme justifiés
3.x — Information par la couleur	Le feedback bonne/mauvaise réponse en session de révision s'appuie sur vert/rouge (section 4.4) ; à doubler d'une information textuelle systématique (déjà partiel : score et correction affichés)	✅ Vérifié conforme : chaque feedback couleur est doublé d'un texte explicite (« Correct »/« Incorrect », score, correction attendue)
13.x — Zones dynamiques	Aucune région <code>aria-live</code> : les mises à jour dynamiques (toasts, compteurs de session) ne	✅ Corrigé : zones <code>aria-live="polite"</code> toujours montées sur le feedback de session (FlashcardsSessionPage) et le score d'exercice (ExerciseDetailPage) ; les toasts portent nativement <code>role="alert"</code>

Critère (RGAA)	Constat	Correction préconisée
	sont pas annoncées aux lecteurs d'écran	

Mise en conformité : campagne outillée et non-régression

Les chantiers annoncés ont été **réalisés et outillés** (campagne du 2026-07-06, tracée dans [.agents/CHANGELOG_AGENT.md](#)) ; le livrable d'audit complet, avec méthode, chiffres avant/après et commandes de reproduction, est versionné : [docs/AUDIT_RGAA.md](#).

1. **Audit statique développé pour le projet** ([my_memo_master_front/scripts/audit-a11y.mjs](#)) : vérifie sur les 73 fichiers `.vue` les noms accessibles des champs (RGAA 11.1), les `alt` (1.1), les équivalents clavier des éléments cliquables (7.1), les noms des boutons symboles (11.9), la langue (8.3) et les zones `aria-live` (13.x). Première exécution : **135 non-conformités** — 111 champs sans nom accessible, 14 boutons « × » sans nom, 10 éléments cliquables sans clavier.
2. **Campagne de correction intégrale** : `aria-label` en français sur chaque champ (statiques ou dynamiques pour les champs générés en boucle), noms contextuels sur les boutons symboles (« Fermer », « Supprimer la ressource »...), conversion sémantique (lien natif pour les tuiles de tutoriel), pattern ARIA `role="button"/tabindex/@keydown` sur les cartes, accordéons, cellules de calendrier et dropzone. Les motifs légitimes (overlays de fermeture, `@click.stop`) sont **documentés comme exceptions dans l'outil**. Ré-exécution : **0 non-conformité**.
3. **Non-régression** : 4 tests axe-core (DOM réellement rendu) tournent dans la suite Vitest à **chaque push** ([my_memo_master_front/test/a11y/axe.test.js](#)) — l'accessibilité est vérifiée par la CI au même titre que les tests fonctionnels. Deux zones `aria-live` couvrent désormais les feedbacks dynamiques (session de révision, score d'exercice).

Restent hors périmètre de cet audit outillé, documentés dans [docs/AUDIT_RGAA.md](#) §5 : la mesure des contrastes (à faire au navigateur — jsdom ne rend pas les styles) et un test lecteur d'écran réel (protocole NVDA planifié sur les trois parcours critiques).

5.4 Évolutivité du code

- **API versionnée** : toutes les routes sont montées sous `/api/v1` ([app.js](#)) — une v2 pourra coexister sans casser les clients existants ;
- **Schéma de base évolutif** : **61 migrations Sequelize** versionnées ([my_memo_master_api/migrations/](#)) documentent chaque évolution du schéma et se rejouent sur les deux dialectes (SQLite/PostgreSQL) ;
- **Configuration externalisée** : aucun paramètre d'environnement en dur (section 1.1) — la même image Docker sert plusieurs environnements, y compris par injection runtime de la config front (`window.__APP_CONFIG__`, [docker-compose.yml](#) service `front_server`) ;

- **Contrat d'API documenté** : annotations Swagger sur chaque route, servies sur [/api-docs](#) (section 9.3) — un client tiers peut être développé contre le contrat, pas contre le code ;
- **Mémoire du projet** : le dossier [.agents/](#) (conventions, décisions au format Contexte/Décision/Alternative/Conséquences, changelog d'état) est conçu pour qu'un nouveau développeur — humain ou agent — reprenne le travail sans contexte oral. Une synthèse de ce dispositif est publiée dans [docs/MEMOIRE_PROJET.md](#). C'est de l'évolutivité organisationnelle, complémentaire de l'évolutivité technique.

5.5 Traçabilité des versions

L'historique git est structuré par les conventions de commit [\[ADD\]/\[IMP\]/\[REF\]/\[FIX\]](#) (section 1.4), et doublé de deux journaux versionnés : [.agents/CHANGELOG_AGENT.md](#) (état global module par module + une entrée par ticket terminé, avec dette éventuelle) et [.agents/DECISIONS.md](#) (chaque choix structurant avec l'alternative écartée). Les évolutions du prototype sont donc tracées à trois niveaux : le commit (quoi), le changelog (où en est-on), la décision (pourquoi).

Section 6 — Déploiement continu et progressif

Compétence couverte : C2.2.4 — Déployer le logiciel à chaque modification de code et de façon progressive en vérifiant la performance fonctionnelle et technique auprès des utilisateurs afin de présenter une solution stable et conforme à l'attendu.

6.1 Déployer à chaque modification : le protocole

Chaque fusion sur une branche d'intégration déclenche un déploiement automatique complet — à condition que la CI soit verte (mécanisme [workflow_run](#), section 2.4). Le pipeline [.github/workflows/cd.yml](#) enchaîne systématiquement : construction des images Docker, poussée sur Docker Hub avec un nom d'image propre à l'environnement, puis déploiement sur la cible correspondante (VPS test pour [dev](#), Kubernetes preprod pour [staging](#), Kubernetes prod pour [main](#)).

La **progressivité** du déploiement est structurelle, à deux niveaux :

1. **Promotion entre environnements** : une modification atteint d'abord l'environnement de test (fusion dans [dev](#)), y est vérifiée, puis est promue en preprod ([staging](#)), et seulement ensuite en production ([main](#)). Le même code traverse trois déploiements successifs avant d'atteindre les utilisateurs finaux.
2. **Garde-fou de production** : le job de déploiement prod est conditionné à une variable d'activation explicite, en plus de la branche ([cd.yml](#)) :

```
deploy_prod:
  if: needs.push_images.outputs.branch == 'main' && vars.K8S_PROD_ENABLED == 'true'
```

Ce double verrou permet de fusionner sur `main` (et donc de construire les images de prod) sans déployer, tant que la mise en production n'est pas explicitement décidée — utile pendant la phase de préparation du cluster dédié.

6.2 Des artefacts de déploiement reproductibles

Les images sont construites en **multi-stage** pour livrer des artefacts minimaux et identiques d'un environnement à l'autre :

- **API** ([my_memo_master_api/Dockerfile](#)) : un stage `deps` installe les dépendances natives avec la toolchain de compilation (python3/make/g++, requis par le runtime ONNX du modèle d'IA), puis le stage `production` ne reçoit que les `node_modules` compilés — **aucun outil de build dans l'image finale**. Le commentaire d'en-tête trace la contrainte : « node:22-bookworm-slim est basé sur Debian (glibc) — requis par onnxruntime-node » ;
- **Front** ([my_memo_master_front/Dockerfile](#)) : stage 1 build Vite, stage 2 nginx statique avec `HEALTHCHECK` embarqué.

Au démarrage, l'API applique elle-même ses migrations et ses seeds de référence ([my_memo_master_api/entrypoint.sh](#)) :

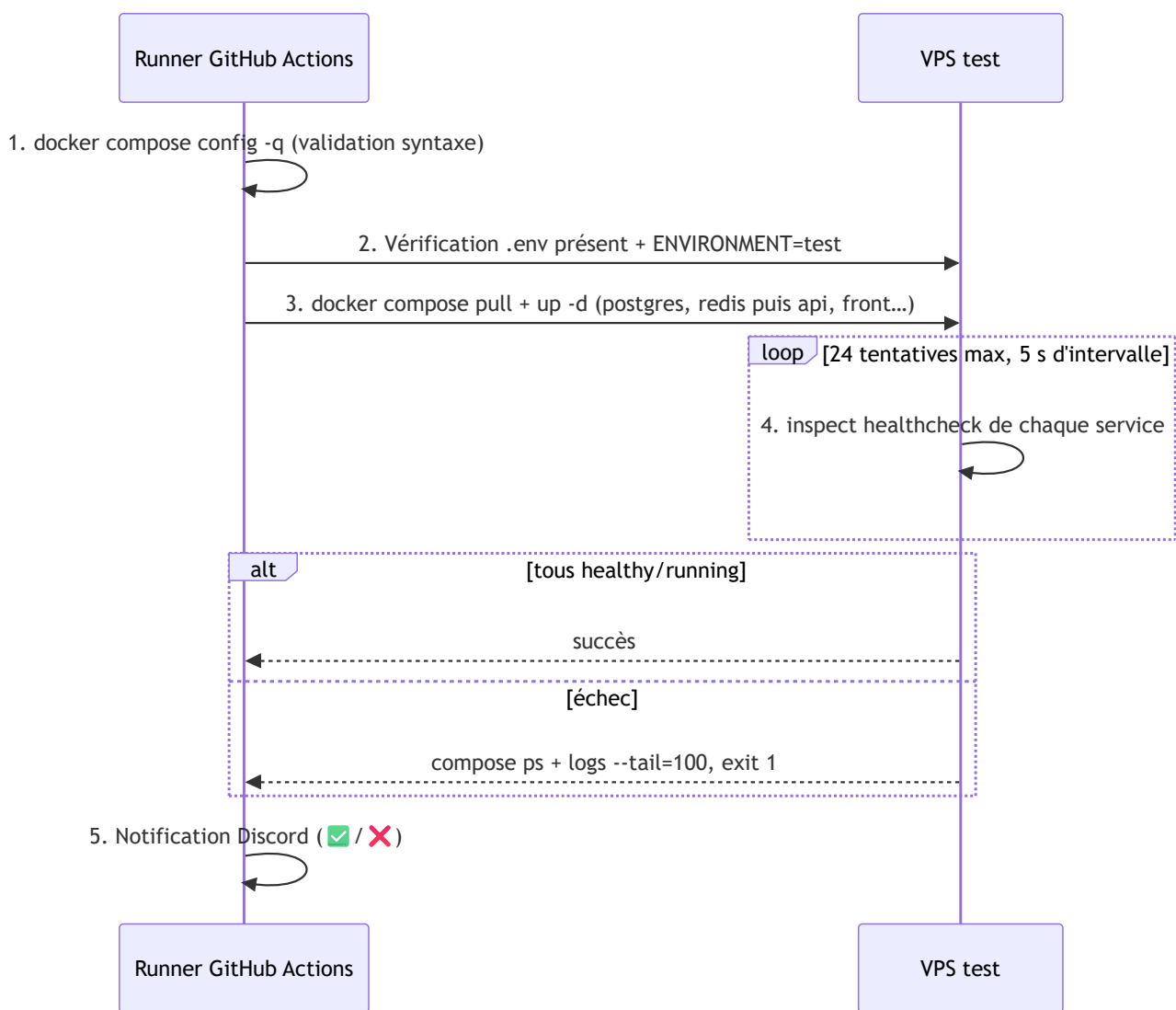
```
echo "[entrypoint] Running database migrations..."
npx sequelize-cli db:migrate

echo "[entrypoint] Seeding roles..."
npx sequelize-cli db:seed --seed 20260605000001-seed-roles.js
```

Le déploiement d'une modification de schéma ne demande donc **aucune intervention manuelle** : la migration voyage avec le code (61 migrations versionnées, section 5.4), et `db:migrate` est idempotent — les migrations déjà appliquées sont ignorées.

6.3 Vérifier avant, pendant et après : le déploiement VPS (test)

Le job `deploy_test` de [cd.yml](#) illustre la démarche « vérifier à chaque étape » :



- **Avant** : la syntaxe du compose est validée sur le runner (`config -q`), puis le `.env` du serveur est contrôlé — présence et cohérence d'environnement (`grep -q '^ENVIRONMENT=test$'`) — ce qui interdit de déployer la configuration de test sur une autre cible ;
- **Pendant** : les services de données démarrent d'abord (PostgreSQL, Redis), les applicatifs ensuite, en s'appuyant sur les healthchecks (section 1.3) ;
- **Après** : une boucle de vérification interroge l'état de santé réel de chaque conteneur (jusqu'à 24×5 s) ; en cas d'échec, le job **collecte les logs** (`compose logs --tail=100`) et échoue explicitement — le déploiement n'est déclaré réussi que si l'application répond, pas seulement si les commandes se sont exécutées.

6.4 Déploiement Kubernetes : atomicité et rollback automatique

Pour la preprod et la prod, le déploiement passe par Helm avec deux mécanismes de stabilité ([cd.yml](#)) :

```

helm upgrade --install mmm-preprod ./helm \
  -f helm/values-preprod.yaml \
  -n mymemomaster-preprod \
  --create-namespace \
  --set "rolloutTimestamp=$(date +%s)" \

```

```
--atomic \  
--timeout 5m
```

- **--atomic** : si le rollout ne converge pas dans le délai (5 min en preprod, 8 min en prod), Helm **annule et restaure automatiquement la version précédente** — un déploiement raté ne laisse jamais l'environnement dans un état intermédiaire ;
- **rolloutTimestamp** : les images étant taguées **latest** par environnement, cette annotation injectée dans le template de pod ([helm/templates/deployment-api.yaml](#)) change à chaque exécution et force un rolling update même à tag constant, combinée à **imagePullPolicy: Always** ;
- **Readiness probes** : le chart déclare des sondes de disponibilité — HTTP / sur le front, HTTP **/api/v1/health** sur l'API — qui conditionnent la bascule du trafic vers les nouveaux pods.

Anomalie détectée lors de la rédaction de ce dossier : en auditant le code pour adosser cette section à des preuves, j'ai constaté que la route **/api/v1/health** ciblée par la readiness probe de l'API **n'existe pas dans le code Express** — le handler 404 global y répond, ce qui empêche les nouveaux pods d'être déclarés prêts et fait échouer le rollout **--atomic**. Ce constat recoupe les échecs de déploiement preprod en cours d'investigation (commits **[TEST] debug pods/pvc/events**, **[TEST] debug node taints/conditions** visibles dans l'historique git). Le correctif — exposer un endpoint de santé minimal dans l'API — est traité dans le plan de correction des bogues (section 8), où cette anomalie sert d'exemple complet du processus de détection/correction.

6.5 Vérification fonctionnelle auprès des utilisateurs

La chaîne d'environnements sert précisément à cela : l'environnement de **test** (**test.my-memo-master.com**) est le lieu de vérification fonctionnelle par les utilisateurs du projet avant promotion — les domaines par environnement sont configurés dans les **.env** serveur et values Helm (sections 1.2, [README.md](#) partie 3). Chaque déploiement notifie le canal Discord de l'équipe ( **Déploiement **staging** réussi.**), qui sait ainsi quand une version est disponible pour vérification.

[SCREENSHOT ICI : environnement de test test.my-memo-master.com et environnement de preprod preprod.my-memo-master.com côte à côte, montrant la même fonctionnalité en cours de validation]

Section 7 — Cahier de recettes

Compétence couverte : C2.3.1 — Élaborer le cahier de recettes en rédigeant les scénarios de tests et les résultats attendus afin de détecter les anomalies de fonctionnement et les régressions éventuelles.

7.1 Démarche : un cahier de recettes exécutable

J'ai fait le choix d'un cahier de recettes **exécutable** : chaque scénario de recette est formalisé sous forme d'un test fonctionnel automatisé versionné dans le dépôt, et le cahier ci-dessous en est la vue documentaire — chaque ligne référence le test qui l'implémente. Ce choix a une conséquence forte sur la détection des régressions : la recette n'est pas rejouée une fois par livraison, elle est rejouée à **chaque push** par la CI (section 2), et un scénario qui casse bloque le déploiement.

Le cahier s'organise en trois volets, conformément au critère (tests fonctionnels, structurels, de sécurité) :

- **Recette fonctionnelle de bout en bout** : les parcours utilisateur complets, implémentés dans [my_memo_master_api/test/bdd/](#) — requêtes HTTP réelles traversant toutes les couches jusqu'à la base (section 4.2) — et, côté interface, dans les tests de pages ([my_memo_master_front/test/components/](#)) ;
- **Recette structurelle** : le contrat HTTP de chaque endpoint (codes de statut, corps, cas d'erreur), implémenté par les 791 tests de [test/controllers/](#) couvrant les 34 modules de l'API — l'exhaustivité des fonctionnalités attendues est portée par ce volet ;
- **Recette de sécurité** : les tests dédiés de [test/middlewares/security.test.js](#) et l'audit OWASP (section 5.2).

Statut de la recette : les statuts «  » des tableaux ci-dessous correspondent à la dernière exécution réelle des suites sur le code actuel du dépôt (1 447/1 447 tests API et 562/562 tests front passés — section 4.1).

7.2 Recette fonctionnelle — parcours de bout en bout

Chaque parcours ouvre sur la user story qu'il valide, pour tracer le besoin utilisateur jusqu'au scénario de test. Les intitulés de scénario et les résultats attendus ci-dessous sont **directement extraits des tests du dépôt** (les intitulés encodent systématiquement condition → résultat attendu, convention de nommage du harnais, section 4.3).

Fonctionnalité cœur — Session de révision Leitner ([test/bdd/leitner.session.test.js](#))

En tant qu'étudiant, je veux réviser mes cartes par répétition espacée — les cartes échouées revenant plus souvent — afin de mémoriser durablement.

ID	Scénario	Résultat attendu	Statut
L-01	Récupération des cartes dues — carte jamais révisée (<code>next_review_at</code> null)	Carte retournée comme due	✓
L-02	Récupération des cartes dues sans token	401	✓
L-03	Bonne réponse à une carte	Carte avance en boîte 2, compteurs mis à jour	✓
L-04	Dépassement de <code>next_review_at</code>	La carte redevient disponible	✓
L-05	Mauvaise réponse	Carte retourne en boîte 1, <code>incorrect_count</code> incrémenté	✓
L-06	Bonne réponse en boîte 5 (dernière)	Reste en boîte 5 (plafonnement), <code>fifo=false</code>	✓
L-07	Session multi-cartes	Plusieurs cartes dues retournées ensemble	✓
L-08	Réponse sur carte inexistante	404	✓
L-09	Réponse sans token	401	✓
L-10	Corps invalide (<code>cardId</code> manquant)	400	✓

Autres parcours — synthèse

Les trois autres recettes fonctionnelles suivent le même formalisme (scénarios extraits des tests : cas nominal + cas limites + erreurs attendues) ; elles sont synthétisées ici, le détail scénario par scénario restant lisible dans les fichiers de test référencés :

Parcours (user story)	Scénarios couverts	Tests	Statut
Passage d'une série d'exercices — <i>en tant qu'étudiant, je veux m'entraîner sur des exercices corrigés automatiquement</i>	Parcours nominal 4/4 avec persistance du score ; 0/4 ; question non répondue traitée comme incorrecte ; correction par	test/bdd/exercise.session.test.js (E-01 → E-10)	✓ 10/10

Parcours (user story)	Scénarios couverts	Tests	Statut
<i>afin d'évaluer ma progression</i>	type de question (QCM, texte à trou insensible à la casse/espaces, remise en ordre) ; erreurs 400/401/404		
Échéances et rappels — <i>en tant qu'étudiant, je veux planifier des échéances avec rappels par email afin d'anticiper mes révisions</i>	Création d'un rappel → job BullMQ planifié ; date déjà passée refusée (400) ; liste avec statut pending ; suppression annulant le rappel et le job ; erreurs 400/401/404	test/bdd/deadline.reminder.test.js (D-01 → D-07)	 7/7
Interface de session de révision — <i>en tant qu'étudiant, je veux un retour immédiat sur chaque réponse pendant ma session de révision</i>	Chargement, session vide, activation du bouton Valider, feedback bonne/mauvaise réponse, fin de session, sortie avec confirmation	test/components/FlashcardsSessionPage.test.js (UI-01 → UI-06)	 6/6

7.3 Recette de sécurité

Volet exécutable ([test/middlewares/security.test.js](#), intitulés réels) :

ID	Scénario	Résultat attendu	Statut
S-01	Requête depuis une origine CORS configurée	Autorisée	

ID	Scénario	Résultat attendu	Statut
S-02	Requête depuis une origine inconnue	Pas d'en-tête <code>access-control-allow-origin</code>	✓
S-03	Preflight OPTIONS	Méthodes et en-têtes autorisés retournés (Content-Type, Authorization)	✓
S-04	5 tentatives de connexion échouées	429 sur /users/login (authLimiter)	✓
S-05	10 inscriptions successives	429 sur /users/register (registerLimiter)	✓
S-06	200 requêtes rapprochées sur une route	429 (apiLimiter global)	✓
S-07	JWT invalide ou expiré	401 (test/middlewares/Auth.middleware.test.js)	✓
S-08	Accès à une route RBAC sans le rôle requis	403 (test/middlewares/requireRole.middleware.test.js)	✓

Volet manuel : l'audit OWASP Top 10 (section 5.2, [docs/SECURITY_AUDIT_OWASP.md](#)) constitue la recette de sécurité approfondie ; ses constats et leur traitement alimentent le plan de correction (section 8).

7.4 Surveillance des régressions

Le dispositif de détection fonctionne en continu :

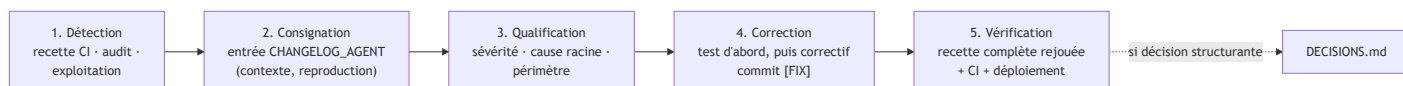
1. **Détection automatique** : la recette exécutable tourne à chaque push (CI bloquante) ; toute régression sur un scénario ci-dessus fait échouer le pipeline **avant** tout déploiement (section 2.4) ;
2. **Signalement immédiat** : l'échec est notifié sur Discord avec la branche fautive (section 2.5) ;
3. **Régression réelle documentée** : ce dispositif a fonctionné en conditions réelles — exemple tracé dans [.agents/CHANGELOG_AGENT.md](#), entrée du 2026-07-04 : un commit d'amélioration (`[IMP] classgroup`) a cassé le comportement du sélecteur de vue de ClassroomPage ; la CI front l'a détecté via `test/components/ClassroomPage.test.js`, et la correction est documentée (analysée en section 8) ;
4. **Anomalies hors recette** : les anomalies découvertes autrement (audit, exploitation) rejoignent le même circuit de consignation et de correction — section 8.

Section 8 — Plan de correction des bogues

Compétence couverte : C2.3.2 — Élaborer un plan de correction des bogues à partir de l'analyse des anomalies et des régressions détectées au cours de la recette afin de garantir le fonctionnement du logiciel conformément à l'attendu.

8.1 Le processus de correction

Le circuit de traitement d'une anomalie suit cinq étapes outillées :



- **Détection** : trois sources alimentent le circuit — la recette exécutable en CI (section 7.4), les audits (OWASP, revue de code), et l'exploitation (échecs de déploiement, logs) ;
- **Consignation** : chaque anomalie corrigée donne lieu à une entrée structurée dans [.agents/CHANGELOG_AGENT.md](#) — contexte permettant de reproduire, fichiers touchés, correction, état résultant. C'est la « fiche d'anomalie » du projet ;
- **Correction** : le correctif est accompagné d'un test qui matérialise le bogue (et empêche sa réapparition), puis commité avec le préfixe `[FIX]` — l'historique en compte **250** (`git log --all --grep="[FIX]"`), ce qui rend le flux de correction auditable ;
- **Vérification** : la recette complète est rejouée localement puis par la CI, qui conditionne le redéploiement (section 2.4) — le correctif emprunte exactement le même chemin qu'une fonctionnalité.

Les trois cas suivants, tous réels et tracés dans le dépôt, illustrent le processus sur les trois sources de détection.

8.2 Cas 1 — Régression fonctionnelle détectée par la recette (CI)

Anomalie (consignée le 2026-07-04, [.agents/CHANGELOG_AGENT.md](#)) : le commit `f4d654e` (`[IMP] classgroup`) a modifié `ClassroomPage.vue` — le sélecteur de vue s'affichait désormais pour les étudiants/enseignants (qui n'en ont pas l'usage) et **disparaissait pour l'admin plateforme** (qui en a besoin).

Détection : la recette front en CI — les tests de [test/components/ClassroomPage.test.js](#) ont échoué sur la branche, avant toute fusion.

Analyse (extrait de la fiche de consignation réelle) : la condition d'affichage `v-if="isAdmin"` avait été remplacée par `v-if="availableViews.length > 1"`, et le calcul de `availableViews` rendu exclusif — l'entrée du changelog décrit précisément le mécanisme de la régression pour chaque rôle, ce qui constitue les « informations permettant de reproduire le bogue » exigées par le critère.

Correction : restauration de la condition et du calcul additif des vues ; vérification par la suite front complète (548/548 verts, consigné dans la fiche).

8.3 Cas 2 — Anomalie d'exploitation : l'échec des déploiements preprod

Ce cas déroule le processus complet sur une anomalie **détectée en exploitation** et corrigée pendant la rédaction de ce dossier (fiche complète : entrée du 2026-07-06 de [.agents/CHANGELOG_AGENT.md](#)).

Symptôme : les déploiements Kubernetes preprod échouaient — rollout `helm --atomic` en timeout puis rollback automatique.

Investigation : instrumentation temporaire du pipeline CD pour collecter l'état du cluster à chaque tentative (étape de debug envoyant pods/PVC/events puis taints/conditions des nœuds vers Discord) — démarche visible dans l'historique git : commits `372529c [TEST] ajoute etape debug temporaire pods/pvc/events sur Discord`, `db59e33 [TEST] debug node taints/conditions....`. On y voit aussi une première hypothèse corrigée en passant : `585ec4d [FIX] remplace le flag helm inexistant --rollback-on-failure par --atomic`.

Cause racine : la readiness probe de l'API cible `GET /api/v1/health` ([helm/templates/deployment-api.yaml](#)), mais cette route **n'existait pas** dans l'API — le handler 404 global répondait, les pods n'étaient jamais `Ready`, le rollout ne convergeait jamais. L'anomalie est structurelle (incohérence chart Helm ↔ code applicatif), pas environnementale — ce qui explique que l'investigation côté cluster (taints, PVC...) ne pouvait pas aboutir.

Correctif (commit `515bf84 [FIX] readiness probe K8s – ajout endpoint /api/v1/health`) : ajout de la route dans [my_memo_master_api/app.js](#) —

```
// Health endpoint – ciblé par les readiness probes K8s (helm/templates/deployment-api.yaml)
// CHOIX: déclaré avant le routeur v1 pour échapper au rate limiting global
app.get('/api/v1/health', async (_req, res) => {
  try {
    await instance.authenticate()
    res.status(200).json({ status: 'ok' })
  } catch (error) {
    logger.error(`Health check – base de données injoignable : ${error?.message || error}`)
    res.status(503).json({ status: 'unavailable' })
  }
})
```

Deux choix de conception accompagnent le correctif, consignés dans [.agents/DECISIONS.md](#) (entrée 2026-07-06) : la route est déclarée **hors du rate limiter** (une sonde kubelet bloquée par le limiteur ferait passer les pods NotReady en cascade — l'alternative `routes/Health.routes.js` est écartée avec justification), et elle teste réellement la base (`authenticate()`) pour qu'un pod sans base de données ne reçoive pas de trafic.

Vérification : 3 tests ajoutés ([test/controllers/Health.test.js](#) — 200 si base joignable, 503 sinon, accès sans authentification), suite complète rejouée : 1 422/1 422 verts. Le correctif est déployé par le pipeline CD standard, et l'étape de debug temporaire du CD a été **retirée** une fois le correctif en place — le pipeline est revenu à son état nominal.

Ce que ce cas démontre : le traitement « tire profit du processus d'intégration et de déploiement continu » (critère C2.3.2) à double titre — l'instrumentation du pipeline a servi d'outil de diagnostic, et le correctif a suivi le circuit standard test → CI → CD sans procédure d'exception.

8.4 Cas 3 — Plan de correction issu d'un audit : les vulnérabilités OWASP

L'audit de sécurité (section 5.2) a produit un **plan de correction priorisé par sévérité**, dont l'exécution est tracée dans [docs/SECURITY_AUDIT_OWASP.md](#) :

Lot	Contenu	Statut
Immédiat (session d'audit, M-00b.07)	8 corrections : routes sans auth (critique), énumération d'emails, tokens non-crypto, Swagger en prod...	✅ Corrigé
Haute priorité avant production (M-00b.07b)	Refresh token hashé, JWT 15 min, contrôle de propriétaire sur Storage.delete, expiration des codes email	✅ Corrigé (tracé par ticket)
Priorité moyenne (2026-07-06)	Magic bytes sur uploads, journalisation des échecs d'authentification et refus d'accès, CSP explicite, anti log-injection, doublon email en 400, caractère spécial mot de passe	✅ Corrigé (tracé dans l'audit)
Risque résiduel assumé	Révocation JWT (blacklist Redis) — palliatif : expiration 15 min + rotation refresh token	📄 Documenté avec recommandation

Ce plan illustre la **qualification** : chaque constat porte une sévérité, un fichier, un correctif proposé — et les éléments non corrigés immédiatement ne sont pas perdus, ils constituent un backlog argumenté avec palliatif documenté (ex. : pas de révocation JWT, palliatif = expiration 15 min).

Section 9 — Documentation technique d'exploitation

Compétence couverte : C2.4.1 — Rédiger la documentation technique d'exploitation du logiciel détaillant son fonctionnement afin d'assurer une traçabilité pour le suivi des équipes et des futures évolutions du logiciel.

9.1 Cartographie de la documentation

La documentation du projet est **versionnée avec le code** — elle évolue dans les mêmes pull requests, ce qui la protège de l'obsolescence. Chaque document a un public et une question à laquelle il répond :

Document	Fichier(s)	Public	Question traitée
Manuel d'installation / prise en main dev	README.md (parties 1–2)	Nouveau contributeur	« Comment je démarre ? »
Manuel de déploiement	docs/MANUEL_DEPLOIEMENT_VPS.md , docs/MANUEL_DEPLOIEMENT_KUBERNETES.md , README.md (partie 3 : secrets CI/CD), docs/https-setup.md	Ops / mainteneur	« Comment je mets en place un environnement ? »
Manuel d'exploitation et de mise à jour	docs/RUNBOOK.md	Ops / astreinte	« Comment j'opère, je mets à jour, je restaure ? »
Manuel d'utilisation	docs/MANUEL_UTILISATION.md	Utilisateur final (étudiant, enseignant, gérant)	« Comment j'utilise l'application ? »
Contrat d'API	Swagger généré, servi sur /api-docs	Développeur front / tiers	« Que fait chaque endpoint ? »
Fonctionnement des modules	docs/DOC_administration_etablissements.md , .agents/DOC_mindmap_editor.md , diagrams/	Développeur reprenant un module	« Comment ce module fonctionne-t-il et pourquoi ? »
Journal des décisions	.agents/DECISIONS.md	Toute l'équipe, futur soi	« Pourquoi le code est-il comme ça ? »
Journal d'état	.agents/CHANGELOG_AGENT.md	Toute l'équipe	« Où en est chaque module ? »

Document	Fichier(s)	Public	Question traitée
Conventions	.agents/CONVENTIONS.md , .agents/AGENT.md	Contributeur (humain ou agent IA)	« Comment on écrit le code ici ? »
Synthèse de la mémoire du projet	docs/MEMOIRE_PROJET.md	Jury, nouveau venu	« Quel est ce dispositif et que contient-il ? »

9.2 Manuel de déploiement

Le déploiement est documenté par deux manuels dédiés, un par type d'infrastructure :

- [docs/MANUEL_DEPLOIEMENT_VPS.md](#) — environnement de **test** (VPS, Docker Compose, Traefik) : architecture déployée, prérequis, préparation initiale du **.env** serveur, premier déploiement manuel, déroulé exact du déploiement continu (validations, boucle de vérification santé), procédure de secours et vérifications post-déploiement ;
- [docs/MANUEL_DEPLOIEMENT_KUBERNETES.md](#) — environnements **preprod et prod** (Kubernetes Infomaniak, Helm) : structure du chart et des fichiers de valeurs, ségrégation ConfigMap (non sensible, versionnée) / Secret (sensible, créé manuellement sur le cluster — « ne jamais committer »), adoption Helm de ressources existantes ([helm-migrate.sh](#)), déploiement continu ([helm upgrade --atomic](#), rolling update forcé), modification de configuration, rollback ([helm rollback](#)) et tableau de correspondance des deux environnements.

La partie 3 du [README.md](#) complète ces manuels sur le volet CI/CD : table de correspondance branche → images → cible, liste exhaustive des secrets GitHub Actions avec leur rôle, procédure d'obtention et d'encodage du kubeconfig Infomaniak. Ces documents décrivent les choix opérés (critère C2.4.1) et pas seulement les commandes : par exemple la ségrégation configuration/secrets, ou le choix **--atomic** qui garantit qu'un déploiement raté restaure automatiquement la révision précédente.

9.3 Manuel d'exploitation et de mise à jour

Le [docs/RUNBOOK.md](#) est le manuel d'exploitation de l'environnement VPS. Sa table des matières témoigne de sa couverture opérationnelle réelle : Prérequis VPS · Premier déploiement · **Mise à jour (déploiement continu)** · **Rollback** · Logs · **Sauvegarde et restauration PostgreSQL** · Commandes utiles · Variables d'environnement critiques · Accès PgAdmin · Surveillance · Contacts et ressources.

Trois points saillants pour la mise à jour :

- le cas nominal est **automatique** (pipeline CD, section 6) — le RUNBOOK documente le mode manuel de secours et le rollback ;
- les migrations de schéma sont documentées des deux côtés : exécution manuelle (`npx sequelize-cli db:migrate`, RUNBOOK § Premier déploiement) et automatique (entrypoint.sh, section 6.2) ;
- la migration d'infrastructure elle-même est outillée et documentée : scripts [k8s/helm-migrate.sh](#) / [k8s/helm-migrate.ps1](#) pour adopter dans Helm des ressources déployées auparavant en `kubectl apply`, référencés en commentaire du pipeline CD au moment exact où l'opérateur en a besoin (`cd.yml` : « Première fois : exécuter k8s/helm-migrate.sh preprod... »).

9.4 Contrat d'API : documentation générée depuis le code

La documentation de l'API est générée par `swagger-jsdoc` à partir des annotations JSDoc portées par **chaque route** (convention illustrée en section 5.1 ; règle de méthode : la documentation Swagger fait partie de la *definition of done* d'une fonctionnalité, [README.md](#) étape 3). La configuration ([my_memo_master_api/config/swagger.config.js](#)) définit le schéma d'authentification et l'applique globalement :

```
components: {
  securitySchemes: {
    bearerAuth: {
      type: 'http',
      scheme: 'bearer',
      bearerFormat: 'JWT',
      description: 'Token JWT obtenu via POST /api/v1/users/login'
    }
  }
},
// CHOIX: sécurité globale plutôt que par route
// RAISON: évite de dupliquer security: [{bearerAuth: []}] sur chaque route protégée
security: [{ bearerAuth: [] }]
```

L'interface interactive est servie sur [/api-docs](#) en développement et test, et **désactivée en production** (correctif F-08 de l'audit OWASP, section 5.2). Générer le contrat depuis le code élimine la principale cause de mort des documentations d'API : la divergence avec l'implémentation.

[SCREENSHOT ICI : interface Swagger UI sur [/api-docs](#) montrant la liste des endpoints et le schéma `bearerAuth`]

9.5 Traçabilité pour les équipes et les évolutions futures

Le critère « assurer une traçabilité pour le suivi des équipes et des futures évolutions » est traité par un dispositif que j'ai conçu explicitement pour la **reprise sans contexte oral** — le dossier [.agents/](#) :

- [.agents/DECISIONS.md](#) impose un format à chaque décision structurante — *Contexte / Décision / Alternative écartée / Conséquences* — dont l'objectif est énoncé dans son en-tête : « permettre de comprendre "pourquoi" le code est comme il est, 2 mois plus tard ». Les sections précédentes de ce dossier en ont cité plusieurs entrées réelles (choix SQLite/PostgreSQL, rotation des refresh tokens, RBAC en base, placement du health endpoint) ;
- [.agents/CHANGELOG_AGENT.md](#) maintient un tableau d'état module par module et une entrée par ticket (fichiers touchés, ce qui est utilisable, hypothèses, **dette éventuelle**) — les fiches d'anomalie de la section 8 en sont des extraits ;
- la documentation de fonctionnement par module descend au niveau des règles métier : [docs/DOC_administration_etablissements.md](#) (11 sections : modèle de données, rôles et droits, référence des endpoints, audit trail, flux complets, contraintes et gardes, **dette connue et limitations V1**), [diagrams/schema_bdd.md](#) (ERD), [diagrams/leitner_algo.md](#), [diagrams/mindmap_rules.md](#), etc.

Documenter la **dette et les limitations** dans les livrables (et pas seulement les réussites) est un choix délibéré : c'est ce qui rend la documentation digne de confiance pour celui qui reprend le code.

9.6 Manuel d'utilisation

L'aide à l'utilisateur final existe à deux niveaux complémentaires. **Embarquée dans l'application** : parcours d'onboarding à la première connexion (page dédiée + état persisté côté serveur, module [OnboardingState](#)), page de tutoriels alimentée par l'API (module [Tutorials](#), [my_memo_master_front/src/pages/TutorialsPage.vue](#)) — le guidage contextuel est adapté à une application grand public. **Manuel autonome** : [docs/MANUEL_UTILISATION.md](#), versionné avec le code, couvre les trois profils (étudiant, enseignant, gérant d'établissement) : création de compte et cycle de session, flashcards Leitner, cartes mentales, exercices, organisation (calendrier/to-do/rappels), suivi de progression et consentement KPI, groupes classes, administration, et une FAQ des blocages courants (email non vérifié, rate limiting, formats d'upload).

[SCREENSHOT ICI : parcours d'onboarding et page Tutoriels dans l'application]

Section 10 — Glossaire

Terme	Définition dans le contexte du projet
BullMQ	File de tâches asynchrones sur Redis ; utilisée pour planifier l'envoi des rappels par email.
CD (déploiement continu)	Pipeline qui construit les images et déploie automatiquement à chaque fusion validée (cd.yml).
Chart Helm	Paquet de templates Kubernetes paramétrés ; un seul chart sert la preprod et la prod via des fichiers de valeurs.

Terme	Définition dans le contexte du projet
CI (intégration continue)	Validation automatique (tests, lint, build) de chaque push (ci.yml).
CORS	Mécanisme du navigateur contrôlant les origines autorisées à appeler l'API ; configuré en liste blanche.
Harnais de test	Ensemble des tests automatisés et de leur outillage d'exécution (Jest/Supertest, Vitest).
Healthcheck / readiness probe	Sonde vérifiant qu'un conteneur/pod est apte à recevoir du trafic ; conditionne démarrages et rollouts.
JWT (access token)	Jeton signé porté par chaque requête authentifiée ; durée de vie 15 minutes.
Leitner (système de)	Méthode de répétition espacée : les cartes montent de boîte en cas de succès et redescendent en boîte 1 en cas d'échec.
Migration (Sequelize)	Script versionné décrivant une évolution du schéma de base de données, rejouable et idempotent.
Monorepo	Dépôt unique contenant API, front, infrastructure et documentation.
Multi-stage (Docker)	Construction d'image en étapes : les outils de build n'entrent pas dans l'image finale.
ORM	Couche d'abstraction de la base de données (Sequelize) ; requêtes paramétrées, dialectes SQLite/PostgreSQL.
OWASP Top 10	Référentiel des dix principales catégories de vulnérabilités web ; base de l'audit de sécurité du projet.
PWA	Progressive Web App : application web installable sur mobile avec manifeste et service worker.
RBAC	Contrôle d'accès par rôles ; implémenté par <code>requireRole</code> avec vérification en base à chaque requête.
Rate limiting	Limitation du nombre de requêtes par client (IP ou userId) sur une fenêtre de temps.
Recette exécutable	Cahier de recettes dont chaque scénario est implémenté par un test automatisé rejoué à chaque push.
Refresh token (rotation)	Jeton opaque long servant à renouveler l'access token ; remplacé à chaque utilisation.
RGAA	Référentiel Général d'Amélioration de l'Accessibilité — référentiel d'accessibilité cible du projet.
Rollback	Retour automatique ou manuel à la version précédente après un déploiement échoué (<code>helm --atomic</code> , RUNBOOK).

Terme	Définition dans le contexte du projet
Rolling update	Remplacement progressif des pods d'une version par la suivante, sans interruption de service.
Seed	Jeu de données de référence inséré en base (rôles, compte admin) au premier démarrage.
SQLite in-memory	Base de données jetable en mémoire utilisée par les tests pour l'isolation et la vitesse.
Wireframe (maquette filaire)	Maquette schématique d'un écran spécifiant zones et hiérarchie sans le graphisme final.

Annexes

Les annexes complètent le corps du dossier sans en conditionner la lecture : chaque compétence est démontrée dans sa section. Les fichiers référencés ci-dessous font partie du livrable (le dépôt est remis avec le dossier).

Annexe A — Galerie complète du prototype (16 écrans)

Captures du prototype interactif (voir section 3.1 pour le contexte et la méthode de génération), au format 1440×900, versionnées dans <docs/prototype/captures/> :

#	Écran	Fichier
1	Connexion	01-connexion.png
2	Inscription	02-inscription.png
3	Accueil (tableau de bord)	03-accueil.png
4	Tutoriels	04-tutoriels.png
5	Cartes mentales — liste	05-mindmaps.png
6	Flashcards — liste des systèmes de Leitner	06-flashcards.png
7	Flashcards — gestion des cartes	07-flashcards-gestion-cartes.png
8	Flashcards — session de révision Leitner	08-flashcards-session.png
9	Exercices — liste	09-exercices.png
10	Exercice — détail et réponse	10-exercice-detail.png
11	Classe (groupes)	11-classe.png
12	Calendrier	12-calendrier.png

#	Écran	Fichier
13	To-do list	13-todo.png
14	Progression — KPI personnels	14-progression-kpi.png
15	Profil	15-profil.png
16	Réglages	16-reglages.png

Le prototype lui-même est manipulable : [docs/prototype/MyMemoMaster - Standalone.html](#) (à servir en HTTP local — voir [docs/prototype/README.md](#)).

Annexe B — Captures de l'application déployée

Écrans de l'application réelle sur les environnements déployés, en complément des captures ciblées insérées dans le corps du dossier.

[CAPTURES ICI : parcours complet sur l'environnement de test — inscription → vérification email → connexion → création d'un système de Leitner → session de révision]

Annexe C — Index des documents du dépôt

Chaque document versionné cité dans le dossier, avec la ou les sections qui s'y appuient :

Document	Chemin dans le dépôt	Utilisé en section
Manuel de déploiement VPS (test)	docs/MANUEL_DEPLOIEMENT_VPS.md	6, 9
Manuel de déploiement Kubernetes (preprod/prod)	docs/MANUEL_DEPLOIEMENT_KUBERNETES.md	6, 9
Manuel d'exploitation (RUNBOOK)	docs/RUNBOOK.md	9
Manuel d'utilisation	docs/MANUEL_UTILISATION.md	9
Mise en place HTTPS (Traefik / Let's Encrypt)	docs/https-setup.md	1, 9
Audit de sécurité OWASP Top 10	docs/SECURITY_AUDIT_OWASP.md	5
Audit d'accessibilité RGAA	docs/AUDIT_RGAA.md	5

Document	Chemin dans le dépôt	Utilisé en section
Synthèse de la mémoire du projet	docs/MEMOIRE_PROJET.md	0, 5, 9
Journal des décisions techniques	.agents/DECISIONS.md	5, 9
Journal d'état (changelog par ticket)	.agents/CHANGELOG_AGENT.md	5, 9
Conventions de code	.agents/CONVENTIONS.md	5, 9
Documentation module Administration établissements	docs/DOC_administration_etablissements.md	9
Documentation technique Éditeur de cartes mentales	.agents/DOC_mindmap_editor.md	9
Schéma de base de données (ERD) et règles métier	diagrams/	0, 3, 9
Pipelines CI / CD	.github/workflows/ci.yml , .github/workflows/cd.yml	2, 6
Chart Helm et manifests Kubernetes	helm/ , k8s/	1, 6

Annexe D — Sources bibliographiques

Études et enquêtes à l'appui du constat de départ (section 0.1 : méthodes de révision passives, préparation tardive) et du choix de méthodes pédagogiques actives (répétition espacée, entraînement par exercices). Fichiers versionnés dans [docs/sources/](#) :

Fichier	Référence	Appuie
2009_Karpicke_Butler_Roediger.pdf	Karpicke, Butler & Roediger (2009), <i>Metacognitive strategies in student learning: Do students practise retrieval when they study on their own?</i> , Memory, 17(4)	Prévalence des stratégies de révision passives (relecture) ; efficacité supérieure de la récupération en mémoire — fondement des systèmes de Leitner
Dunlosky_SciAmMind.pdf	Dunlosky et al., <i>What Works, What Doesn't</i> , Scientific American Mind	Comparaison de l'efficacité des techniques d'étude : entraînement par tests

Fichier	Référence	Appuie
		et répétition espacée en tête, relecture et surlignage en queue
ZIP_2022.pdf	[À COMPLÉTER : référence exacte de l'enquête 2022]	Statistiques citées en section 0.1
Texte+4_pp.79-102.pdf	[À COMPLÉTER : référence exacte, pp. 79–102]	Statistiques citées en section 0.1

Annexe E — Synthèse de couverture des compétences

Récapitulatif de la couverture du référentiel, une ligne par compétence ; les preuves détaillées figurent dans la section correspondante.

Compétence	Section	Réponse apportée (preuves clés)
C2.1.1 — Environnements de développement, test, déploiement	1	Docker Compose à profils + mode hors Docker (SQLite) ; test = VPS Compose, preprod/prod = Kubernetes Helm ; ESLint ×2, Prettier, Jest/Vitest, npm audit bloquant ; healthchecks, métriques Prometheus RED/USE, logs Winston/Morgan ; monorepo GitHub, stratégie de branches alignée sur les environnements
C2.1.2 — Intégration continue	2	Workflow ci.yml (npm ci → test → lint → audit → build), matrice de jobs ciblée par préfixe de branche, harnais hermétique (SQLite in-memory, mocks), CI bloquante en amont du CD (workflow_run), notification immédiate des échecs
C2.2.1 — Prototypage	3	Prototype HTML interactif versionné (14 écrans) + application déployée ; bibliothèque de composants réutilisables ; responsive Tailwind, PWA, onboarding ; guards de navigation, refresh token silencieux, jauge de mot de passe
C2.2.2 — Harnais de tests unitaires	4	2 009 tests verts (1 447 API + 562 front) ; jeu complet sur la fonctionnalité cœur Leitner (21 unitaires + 10 fonctionnels + 13 UI) ; cas nominal/limites/erreurs systématiques ; exécution bloquante en CI à chaque push
C2.2.3 — Développement : évolutivité, sécurisation, accessibilité	5	Architecture en couches uniforme (36 controllers) ; audit OWASP versionné (8 corrections tracées, défense en profondeur) ; RGAA 4 : 135 non-conformités → 0, non-régression axe-core en CI ; API versionnée, 61 migrations ; double journal (état, décisions)

Compétence	Section	Réponse apportée (preuves clés)
C2.2.4 — Déploiement continu et progressif	6	CD automatique conditionné à la CI verte ; promotion test → preprod → prod avec verrou <code>K8S_PROD_ENABLED</code> ; healthchecks bloquants, <code>helm --atomic</code> avec rollback auto ; images multi-stage reproductibles, migrations idempotentes
C2.3.1 — Cahier de recettes	7	Volet structurel : contrat HTTP des 33 modules (785 tests) ; 42 scénarios formalisés avec résultat attendu ; trois volets (fonctionnel, structurel, sécurité) exécutés à chaque push ; régression réelle détectée et tracée (2026-07-04)
C2.3.2 — Plan de correction des bogues	8	3 sources de détection illustrées par 3 cas réels ; fiches de consignation reproductibles (<code>CHANGELOG_AGENT</code>) ; analyse de cause racine ; correctifs avec tests anti-réapparition livrés par le circuit CI/CD ; 250 commits <code>[FIX]</code> auditables
C2.4.1 — Documentation technique d'exploitation	9	Manuels de déploiement (VPS, Kubernetes), d'exploitation (RUNBOOK), d'utilisation (3 profils + FAQ) ; décisions au format Contexte/Décision/Alternative/Conséquences ; contrat d'API Swagger généré depuis le code

Fin du dossier — Bloc 2. Les compétences C2.1.1, C2.1.2, C2.2.1, C2.2.2, C2.2.3, C2.2.4, C2.3.1, C2.3.2 et C2.4.1 sont chacune couvertes par une section dédiée avec preuves issues du dépôt.